

ORIGINAL ARTICLE

Eliminating the high false-positive rate in defect prediction through BayesNet with adjustable weight

Yanyang Zhao^{1,2}  | Yawen Wang^{1,2} | Dalin Zhang³ | Yunzhan Gong¹

¹State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China

²Guangxi Key Laboratory of Cryptography and Information Security, Guilin, China

³School of Software Engineering, Beijing Jiaotong University, Beijing, China

Correspondence

Yawen Wang, State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China.
Email: wangyawen@bupt.edu.cn

Funding information

National Natural Science Foundation of China, Grant/Award Number: U1736110; Guangxi Key Laboratory of Cryptography and Information Security, Grant/Award Number: GCIS202103

Abstract

In defect prediction, a high false-positive rate (FPR) caused by class imbalance not only increases the workload of testing and development but also consumes unnecessary costs. Many defect models against class imbalance have been proposed to improve the accuracy of defect prediction, but their ability to reduce FPR is unclear. To solve these problems, we first proposed a BayesNet with adjustable weights, called WBN, to reduce the FPR in software defect prediction, which is an algorithm independent of data preprocessing techniques. The mechanism of our WBN is to change the sampling probability of the misclassified instances when training the defect model, making the BayesNet model focus more on false alarm instances. And then, we investigate the FPR of five mainstream defect models for solving class imbalance and select them as comparison models to test the validity of our methods. The experimental result on eight open-source projects shows that a) our WBN, in inversion defect prediction (IVDP) and cross-version defect prediction (CVDP), effectively reduces FPR with means of 0.384 and 0.322, respectively; b) compared with improved subclass discriminant analysis (ISDA) that is the lowest FPR in all control models, our WBN not only reduced the FPR but maintained recall whose mean value was 0.797, whereas ISDA did not, with an average recall of only 0.397; c) our WBN, in CVDP, not only reduces FPR, but also has significant superiority over five control defect models and baseline. Besides, we also found that the class imbalance difference between the test set and the training set has an impact on CVDP performance, recommending that practitioners choose the best dataset for CVDP from the defect data of the historical version through special technology.

KEYWORDS

BayesNet, class imbalance, defect prediction, false-positive rate, machine learning

1 | INTRODUCTION

With the increase of software complexity, the reliability of software testing is increasingly required (H. Gao et al., 2018; Huang et al., 2021). Defect prediction based on learning algorithm is one of the most important methods for software reliability maintenance (Tantithamthavorn et al., 2017). It is well known that the distribution of defects is uneven, where clean classes/models are much larger than defect classes/models,

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2022 The Authors. *Expert Systems* published by John Wiley & Sons Ltd.

which is called a class imbalance. In defect prediction, the class imbalance can cause high false positives and reduce the reliability of the defect model, which will bring unnecessary troubles to the testing work (Mahmood et al., 2015; Thabtah et al., 2020).

To overcome the class imbalance, researchers have put forward many methods, including class rebalancing approaches, learning models, and hybrid methods (Ganganwar, 2012). Class rebalancing technology mainly resamples the instances to make the classes contained in the new dataset balance (Diez-Pastor et al., 2015; Kamei et al., 2007; Thabtah et al., 2020). For example, undersampling technique keeps minority class balanced with majority class by controlling the sampling of majority class (Lopez et al., 2013). These technique is more concerned with size differences between majority and minority classes without focusing on independent instances. That is, it may affect false alarms in defect prediction when balancing training data with sampling techniques, but not deal with the false alarm. Many researchers begin to solve this problem from the learning algorithm level (Arun & Lakshmi, 2020; K. Gao et al., 2014; Guo & Viktor, 2004; Sri Kavya, 2020; S. Wang & Yao, 2013), involving classification-threshold adjustment, cost-sensitive methods, and weight learning models. Compared with these approaches, integrated learning attracts more attention from practitioners, for example, Boosting, Bagging, Vote, and Random Forest (Ozturk, 2017; X. Zhang et al., 2015). Song et al. (2019) validate that Boosting (Freund & Schapire, 1997) and Bagging perform better independent learners (Galar et al., 2012; Lopez et al., 2013), which can improve the classification ability of models. But whether they can effectively reduce the false-positive rate (FPR) has not been verified, which is one of the concerns of our study.

Similar problems arise when implementing virtual diagnosis with deep learning in medicine. To avoid disastrous results from false alarms in medicine, many approaches (e.g., three-way decision-based Bayesian deep learning (TWDBDL) (Abdar, Samami, et al., 2021)) are to provide uncertainty quantification for their output to increase the interpretability of the results (Abdar, Pourpanah, et al., 2021). But most of these techniques are embodied in the area of image processing, where source code defect prediction is not. When considering the differences in technology application scenarios, it is a difficult problem to migrate this strategy to defect prediction.

To solve this problem, the performance of AdaBoost, Bagging, Vote, and Random Forest is examined on the FPR, of which AdaBoost is one of the most famous representatives of Boosting algorithms. Following this, a method of BayesNet with adjustable weights is designed to solve the problem of FPR, which is called WBN. Its special feature is that it can feedback the information of defect prediction to the next stage, which is an algorithm independent of data preprocessing techniques. First, the BayesNet model is trained from the dataset by random sampling, where the probability is the same for every instance employed. According to the accuracy of the training results, that is, the classification error rate, setting a weight coefficient for the model at this stage. Meanwhile, the learning results are fed back to the next stage for adjusting the weight of the sample. And then, a dataset with different sampling weights is obtained to train the new BayesNet models. The process is iterated continuously. After the iteration, according to the weight information of the feedback, multiple BayesNet models are weighted to build an integrated BayesNet which can adjust the weight. In other words, this method can increase the weight of instances of incorrect prediction and decrease the sampling probability of other samples to reduce the FPR in defect prediction, which is different from other algorithms.

To verify the effectiveness of WBN, the performance of the WBN and five control approaches are compared and analysed on in-version defect prediction (IVDP) and cross-version defect prediction (CVDP). Also, the mean of four independent models was used as the baseline, including J48, JRip, Kstar, and SMO. The results show that our method WBN achieves encouraging CVDP and IVDP performance against the total of five control methods, where WBN obtains the lowest FPR among these algorithms. To the best of our knowledge, we are the first to conduct experiments in CVDP and IVDP to reduce FPR by improving machine learning techniques. We expect that we extracted insights on false positives from these experimental results for alerting practitioners and researchers to the dangers of false positives.

In sum, this paper makes the following contributions:

1. A BayesNet with adjustable sample weight is proposed to solve the problem of high FPR in defect prediction;
2. Benchmarks for CVDP and IVDP are established through four independent learning methods;
3. The performance of improved subclass discriminant analysis (ISDA) and four primary classifiers against class imbalance is verified using five popular performance indicators;
4. According to the class imbalance between training and test set, the CVDP experimental pair is divided into three modes, covering imbalance-imbalance, balance-imbalance, and balance-balance, and their work efficiency is revealed through experiments
5. We found that the class imbalance difference between the test and the training set has an impact on CVDP performance, recommending that practitioners choose the best dataset for CVDP from the defect data of the historical version through special technology.

The rest of the paper is organized as follows. In Section 2, the learning algorithms proposed by researchers for solving the class imbalance effect in the defect prediction are reviewed. In Section 3, an application model of the BayesNet method with adjustable weights is presented. In Section 4, experimental settings for IVDP and CVDP are carried out to demonstrate the effectiveness of our method, including experimental scenario, dataset, and performance metrics. In Section 5, the experimental results of IVDP and CVDP are analysed. In Section 6, we discussed the practical implications of WBN to the software engineering community. In Section 7, the universality and limitations of our method are analysed. In Section 8, we summarized our research conclusions and introduced our next work direction.

2 | RELATED WORK

The false alarm problem caused by data quality has attracted more and more attention from researchers in various industries. For example, Li, Meng, et al. (2018) designed a Medical Fuzzy Alarm Filter (named MFALFilter) for reducing false positives using fuzzy logic. Our work is closely related to previous researches on the class imbalance of defect prediction in software testing, for example, sampling processing, cost-sensitive learning, and ensemble learning. In this section, we review the methods for solving class imbalances about defect prediction through different technical routes. We will point out the effects of algorithms on FPR if the study contains it.

Sampling processing is also known as class rebalancing technology, which mainly resamples the instances of a few classes in the dataset to make the classes contained in the new dataset balance (Bennin et al., 2019). For example, Feng et al. (2021) proposed a Complexity-based OverSampling technology to alleviate the class imbalance problem. Bennin et al. (2018) proposed a Diversity-based oversampling approach to moderate the class imbalance issue, called COSTE and MAHAKIL, respectively. Tantithamthavorn et al. (2020) investigated the impact of class rebalancing techniques (i.e., over-sampling, under-sampling, Default SMOTE, Optimized SMOTE, and ROSE techniques) on the performance measures and interpretation of defect models. The experimental results show that the optimized SMOTE technique and the under-sampling technique are beneficial to increase AUC and Recall in defect prediction. Sampling technique is not perfect in practice. For example, the information of training data is diluted; the information between instances of data set is lost and overlapped; the performance of classification model is affected by edge instances (Lopez et al., 2013). Note that the sampling technique (e.g., over-sampling and under-sampling) is a data processing technique, while our method is a classification model independent of the sampling technique. In other words, sampling techniques were not the focus of our study.

To solve the class imbalance in defect prediction, practitioners have designed many cost-sensitive learning approaches. Khoshgoftaar et al. (2002) proposed a cost-Boosting algorithm to resolve class imbalance, where C4.5 and decision stumps were selected as base classifiers. The results on two large-scale industrial software show that Boosting-C4.5 improves the accuracy of the classifier, but other models (including Cost-Boosting) do not under type I&II error. To solve the problem of class imbalance, Siers and Islam (2015) customized a cost-sensitive decision forest algorithm called CSForest, which includes oversampling technology STOME. The cost ratio of this method is TP: TN: FP: FN = 1:0:1:5. The experimental results on six projects from the NASA repository show that the CSForest is a potential solution to class imbalance. Liu et al. (2014) designed a two-stage cost-sensitive learning (TSCS) method for defect prediction, utilizing cost information not only in the classification stage but also in the feature selection stage. Experimental results on seven real data sets from NASA projects suggest that our TSCS method achieves better performance under sensitivity and accuracy compared to existing single-stage cost-sensitive classifiers. There are many methods similar to the above methods, such as cost-sensitive discriminative dictionary learning (CDDL) proposed by Jing et al. (2014). Since class imbalance can affect the accuracy of the evaluation index in machine learning, the reliability of the evaluation metrics they employed (i.e., sensitivity and accuracy) is questioned. Regarding cost-sensitive learning, the core problem is the generalization ability of the algorithm for different projects, that is, the evaluation of value is a challenge, even for experienced experts, which hinders the popularization of the technology in practice (Khoshgoftaar et al., 2002).

Also, ensemble learning is one of the mainstream methods to solve the problem of class imbalance. Chawla et al. (2003) designed a new learning method combining the SMOTE and the Boosting called SMOTEBoost, which is a variant of the AdaBoost.M2 procedure. Experimental results from several imbalanced datasets indicate that the SMOTEBoost can obtain better prediction results about minority classes than AdaBoost, AdaCost, and other classifiers. However, this study only shows that recall, Precision, and F-measure without FPR. Seiffert et al. (2009) analysed class rebalancing technology and Boosting in defect prediction using AUC and K-S, where K-S is the absolute of difference value between TPR and FPR. Their results showed that although the sampling method can effectively improve the performance of the classifier, the Boosting algorithm is better than sampling technology. It also points out that Boosting is not a specially designed method to solve class imbalance, but it is very effective to solve this problem. For training the defect model from imbalance data, Seiffert et al. (2010) presented a new hybrid sampling-Boosting algorithm, called RUSBoost. This paper evaluates the performances of RUSBoost and SMOTEBoost, as well as their individual components (i.e., random undersampling [RUS], SMOTE, and AdaBoost). Experimental results show that RUSBoost, SMOTEBoost, and AdaBoost have better performance than other processes, whereas RUSBoost performs often better than SMOTEBoost. Zheng (2010) studied three value-sensitive Boosting approaches to boost neural networks for software defect prediction, which are called CSBNN-TM, CSBNN-WU1, and CSBNN-WU2, respectively. The CSBNN-TM moved the threshold towards the clean modules, enabling more defect modules can be classified correctly. The other two (i.e., CSBNN-WU1 and CSBNN-WU2) are weight-based updates. The experimental results on four projects from NASA datasets suggest that CSBNN-TM is better than the other two models under the Normalized Expected Cost of Misclassification. Diez-Pastor et al. (2015) proposed a new preprocessing technique RB-Boost using Random Balance and Boosting to balance datasets. The experimental results show that RB-Boost improves AUC significantly, but not F-measure compared with AdaBoostM1, MultiBoost, SMOTEBoost, and RUSBoost. Although this algorithm avoids adjusting the scale parameters of sensitive classes, the advantages of this method are not obvious against baseline methods. And, Wang et al. (2016) obtained the optimal multi-core classifier by Boosting method, which is called MKEL. The experimental results show that MKEL method is better against total six methods under F-1, including coding-based ensemble learning (CEL) (Sun et al., 2012), dynamic AdaBoost. NC (S. Wang & Yao, 2013), weighted Naïve Bayes (T. Wang & Li, 2010), Compressed C4.5 decision tree (Wang et al., 2012), Cost-sensitive

Boosting Neural Network (CBNN) (Zheng, 2010), and Asymmetric Kernel Principal Component Classification (AKPCC) (Y. Ma et al., 2012). As F-1 is problematic in defect prediction (Yao & Shepperd, 2021), the experimental results are sceptical. Most of these classical methods are associated with Boosting, but the evaluation indicators of these methods are not unified, which cannot provide a meaningful reference for industry personnel. In this study, when evaluating the performance of the defect model, we provide five effective and common evaluation indicators for stakeholders to make up for this shortcoming.

Prior studies have also focused on an overview of ensemble learning under class imbalances to provide a reference for practitioners. Galar et al. (2012) reviewed ensemble learning methods for class imbalances: Bagging -, Boosting -, and hybrid based approaches. In the result analysis, it is proposed that Bagging and Boosting have been proved to be very effective for solving the class imbalance problem. The experimental results show that the performance of Boosting is better than Bagging. S. Wang and Yao (2013) studied different types of class imbalance methods, including resampling technology, threshold shift, and integration algorithm (i.e., AdaBoost.NC and SMOTEBoost). Among these algorithms, AdaBoost.NC has better performance than other methods. To improve the performance of AdaBoost.NC, a dynamic version of AdaBoost.NC is proposed, which adjusts its parameters automatically during training. Its experimental results show that the error detection rate of AdaBoost.NC is lower than that of NaïveBayes in 9 out of 10 cases.

Besides, some other factors are considered to solve the problem of class imbalance in defect prediction, for example, feature selection and subclass optimization. Khoshgoftaar et al. (2014) designed a technology combining feature selection and ensemble learning to solve the problems of high-dimensional feature and class imbalance. The results show that Boosting algorithm has better classification performance than the ordinary algorithm under AUC and PRC. For within-project prediction, Jing et al. (2017) ISDA for achieving balanced subclasses, also known as ISDA. Extensive experiments on four widely used datasets indicate that ISDA-based solution performs better than eight state-of-the-art methods, covering support vector machine (SVM), Random Forest, random oversampling (ROS), RUS, TSCS (Liu et al., 2014), CDDL (Jing et al., 2014), CEL (Sun et al., 2012), subclass discriminant analysis (SDA) and AdaBoost.NC (S. Wang & Yao, 2013). Pandey and Tripathi (2021) dealt with the impact of noise and class imbalance problem on five defect models by adding the various noise level (080%), which provides guidelines for the possible range of tolerable noise for baseline models. The experimental results on 864 experiments over three public datasets show that Random forest outperforms compared with other state-of-the-art techniques under AUC, which has a high noise tolerance rate (3040%). Namely, random forest is more stable than other learners when learning from class unbalanced noise data.

Table 1 shows the simple statistics of related work, including comparison method, best methods, evaluation indicators. Through the comparative study of these methods in the Table 1, it is found that ensemble learning is better than other approaches for solving class imbalance problems, where Boosting algorithm family shows superiority. This information provides a reference for our experimental setup. With regard to evaluation metrics, these studies employed metrics that were discrepant, even ignored the false-positive indicators, although these studies are very meaningful to the software engineering community. Galar et al. (2012), Wang et al. (2016), and Pandey and Tripathi (2021) hired only one measure, which did not provide a well-established reference for participants. Khoshgoftaar et al. (2014) and Diez-Pastor et al. (2015) employed comprehensive performance indicators, which makes people confused about basic indicators (e.g., recall and FPR). Besides, the evaluation information provided by Khoshgoftaar et al. (2002) and Zheng (2010) is not intuitive enough. Among the above mentioned studies, only three studies provide reference indicators of false alarms. In defect prediction, FPR is an indispensable and important evaluation index, which is discussed in Section 6. This study will make up for this deficiency.

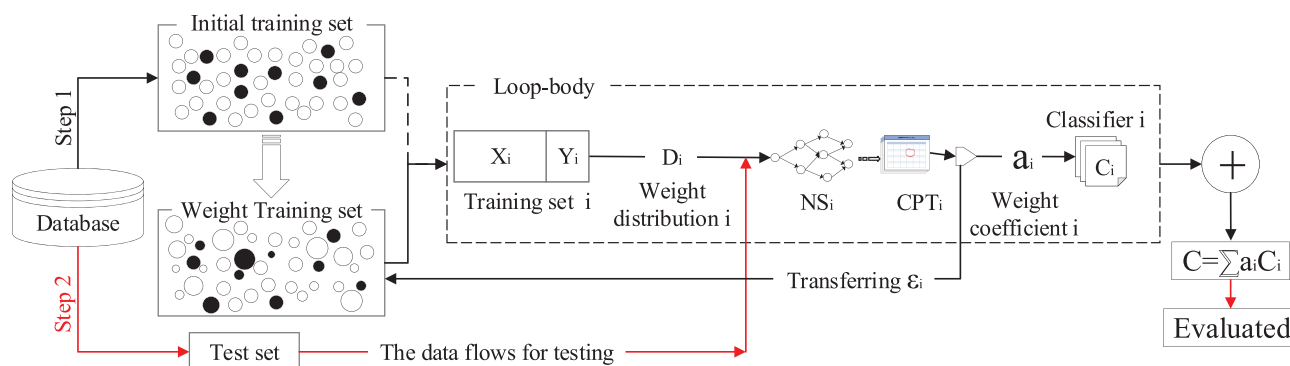
3 | METHOD INTRODUCTION

Figure 1 shows a structure diagram of a BayesNet with adjustable weights, which is called WBN. In the initial and weight training set, the black and white circles represent defects and clean instances, respectively. In the weight training set, the size of the circle indicates the weight, in which the larger circles refer to a greater probability of being sampled. The dotted line surrounds the loop-body of our method WBN. The black solid and black dotted lines represent the training data flow, where the dotted line represents the sampled data flow to the initial training set. The red line represents the data flow for testing. As shown in Figure 1, the WBN principally consists of three steps in each cycle: the first step is to learn the Network structure, which is called NS; the second step is to learn the conditional probability table, known as CPT; the third step yields the weak classifier C_i and the feedback information, covering classification error rate ϵ and weight coefficient of base classifier α . After the loop, a well-established classification model WBN was obtained by combining multiple weak classifiers with weight factors. This method is described in detail as follows.

First, a training set T_t is selected including t defective instances, where the probability of each instance being sampled is the same, namely, the distribution of the weights for initializing the training data is $D_0 = (\omega_{0,1}, \dots, \omega_{0,j}, \dots, \omega_{0,t})$, $\omega_{0,j} = \frac{1}{t}$. The NS is obtained from the instance information in the dataset T_t through K2. The NS is a directed acyclic graph that depicts the dependencies between attributes, which can effectively express conditional independence between instance attributes. In the algorithm flow of K2, as shown in Table 2, function g is an evaluation function to determine whether attribute z is the parent node of x_i , as follows:

TABLE 1 The simple statistics of related work

Research	Method	Control methods	Best method	Performance indicators
Khoshgoftaar et al. (2002)	Cost boosting	Boosting-C4.5; boosting-decision stumps	Boosting-C4.5	Confusion matrix
Chawla et al. (2003)	SMOTEBoost	AdaBoost; AdaCost	SMOTEBoost	Recall, precision and F-measure
Seiffert et al. (2009)	Boosting	Class rebalancing technology	Boosting	AUC and K-S
Seiffert et al. (2010)	RUSBoost	SMOTEBoost; random undersampling; synthetic minority oversampling technique; AdaBoost	RUSBoost	K-S; AUC; PRC; F-measure
Zheng (2010)	CSBNN	TM; WU1; WU2	CSBNN-TM	Misclassification rate
Galar et al. (2012)		Bagging; boosting; hybrid-	Boosting	AUC
Wang and Yao (2013)	AdaBoost.NC	Resampling technology; threshold shift; SMOTEBoost; Naive Bayes; random forest;	AdaBoost.NC	Recall; FPR; G-mean; balance;
Khoshgoftaar et al. (2014)	Combining feature selection	RUSBoost; AdaBoost; MLP; SVM	Boosting	AUC; PRC
Liu et al. (2014)	TSCS	CBNN	TSCS	Sensitivity; accuracy
Diez-Pastor et al. (2015)	RB-Boost	AdaBoostM1; MultiBoost; SMOTEBoost; RUSBoost	RB-Boost	AUC; F-measure; G-mean
Siers and Islam (2015)	CSForest	C4.5; SVM; Voting; CSVoting	CSForest	Weighted precision and recall
Wang et al. (2016)	MKEL	CEL; AdaBoost.NC; Naïve Bayes; C4.5; CBNN; AKPCC	MKEL	F-measure
Jing et al. (2017)	ISDA	SVM, random forest; ROS; RUS; TSCS; CDDL; CEL; SDA; AdaBoost.NC	ISDA	Recall, FPR, AUC, F-measure
Tantithamthavorn et al. (2020)		ROS; RUS; SMOTE; ROSE	SMOTE	10 measures (e.g., AUC)
Pandey and Tripathi (2021)		Random forest; AdaBoost; J48; SVM; Naive Bayes	Random forest	AUC

**FIGURE 1** Structure diagram for BayesNet with adjustable weight

$$g(i, \pi_i) = \sum_{j=1}^t \sum_{k=1}^{q_i} \frac{(r_i - 1)!}{(N_{ij} - r_i - 1)!} \sum_{k=1}^{r_i} N_{ijk}! \quad (1)$$

where, i denotes the order of attributes; j denotes the order of the value of the parent node of x_i ; q_i denotes the number of values of the parent node of x_i ; r_i denotes the number of values of the i_{th} attribute; N_{ij} denotes the number of instances corresponding to the j_{th} value of the parent node of the i_{th} attribute; N_{ijk} denotes the number of instances corresponding to the k_{th} value of the i_{th} attribute and the j_{th} value of its parent node.

TABLE 2 Description of K2 algorithm

Input: DataSet T_t to contain t instances, each instance with n attributes, an ordering on the attributes, an upper bound μ on the number of parents a attribute may have. x_i denotes the i_{th} attribute; $Pred(x_i)$ refers to the attributes before x_i in attribute order; $|\pi_i|$ is the number of parent nodes of the i_{th} attribute;

process:

- 1: for $i = 1$ to n do
- 2: $\pi_i = \emptyset$;
- 3: $P_{old} := g(i, \pi_i)$; this function is computed using Equation (1)
- 4: OKToproceed: = ture
- 5: while OKToproceed and $|\pi_i| < \mu$ do
- 6: let z be the attribute in $Pred(x_i) - \pi_i$ that maximizes $g(i, \pi_i \cup \{z\})$;
- 7: $P_{new} := g(i, \pi_i \cup \{z\})$;
- 8: if $P_{old} > P_{new}$ then
- 9: $P_{old} := P_{new}$;
- 10: $\pi_i := \pi_i \cup \{z\}$
- 11: else OKToproceed: = false;
- 12: end {while};
- 13: write (attribute: x_i , parents of this node: π_i)
- 14: end {for}

Output: For each node, a printout of the parents of the node.

After completing the network structure learning, the CPT of the BayesNet is estimated, where each value in the table has an initial value of 0.5. The learning algorithm for the CPT is a simple estimator, which updates the CPT directly from the instance by estimating the probability based on the characteristic network structure. The conditional probability for each attribute x_i is calculated as follows:

$$\theta_{x_i|\pi_i} = P_{D_0}(x_i|\pi_i) \quad (2)$$

The joint probability distribution of all the attributes of the instances in the dataset is calculated as follows:

$$P_{D_0} = \prod_{i=1}^n P_{D_0}(x_i|\pi_i) = \prod_{i=1}^n \theta_{x_i|\pi_i} \quad (3)$$

Once the CPT is calculated, the classification error rate ε_0 can be calculated on the training dataset T_t that have sampling weight distribution D_0 , as following:

$$\varepsilon_0 = P(C_0(d_i) \neq y_i) = \sum_{i=1}^m \omega_{(0,i)} I(C_0(d_i) \neq y_i) \quad (4)$$

where C_0 represents the classifier; d_i represents the i_{th} instance in dataset T_t ; y_i represents the label value of the i_{th} instance; $\omega_{(0,i)}$ represents the weight of instance d_i in the dataset T_t ; $I(*)$ is the function to judge whether the instance is classified correctly.

When considering the weight coefficient α_0 of the classifier C_0 , it should minimize the exponential loss function ℓ_{exp} , as follows:

$$\ell_{exp}(\alpha_0 C_0 | D_0) = E_{d \sim D_0} [\exp(-y_i \alpha_0 C_0(d))] \quad (5)$$

From the solution of the derivative of the exponential loss function ℓ_{exp} , the weight update formula for the classifier C_0 is:

$$\alpha_0 = \frac{1}{2} \log \frac{1 - \varepsilon_0}{\varepsilon_0} \quad (6)$$

The new weight distribution D_1 of training set T_t is as follows:

$$D_1 = (\omega_{1,1}, \dots, \omega_{1,i}, \dots, \omega_{1,t}) \quad (7)$$

where, the weight of the each instance is updated as follows:

$$\omega_{1,i} = \frac{\omega_{m,i}}{\ell_{\exp}(\alpha_0 C_0 | D_0)} \exp(-\alpha_0 y_i C_0(d_i)) \quad (8)$$

Then, the classifier C_1 is trained on the training set T_t with weight distribution D_1 , generating its weight factor α_1 and the new weight distribution D_2 . As shown in Figure 1, this process is continued iterating until the end of the loop-body. After iteration, the multiple BayesNet models C_m are combined to predict defective labels by weighting factor α_m , where m is the number of iterations, as follows:

$$C(d) = \underset{y \in \{-1, 1\}}{\operatorname{argmax}} \operatorname{sign} \left(\sum \alpha_m C_m(d) \right) \quad (9)$$

Unlike the traditional decision function, we employ the weight factor α_m in the decision process for balancing the performance of sub-predictors generated in the process of instance weight collection. The weight factor α_m is generated by the classification error rate of the sub classifier on the training data set, as shown in formula 6. Namely, the higher the accuracy of the sub-classifier, the stronger the decision-making ability of this sub-classifier in the decision function.

From the above description, the WBN we designed can change the sampling probability of the misclassified instances through the classification error rate ε . When an instance is misclassified multiple times, the WBN increases the probability of it being sampled with each learning of the subclassifier. Therefore, WBN can focus more on false alarm instances. Also, considering the correctly classified instances, the WBN can improve the proportion of subclassifiers with higher accuracy using the weighting factor α obtained from the classification error rate ε . That is, the WBN model, when focusing on false-positive instances, simultaneously guarantees the reliability of the model.

4 | EXPERIMENTAL SETUP

To verify the effectiveness of our WBN for mitigating false positives, we have designed experimental cases of defect prediction. This section describes the experimental scenario, comparison models, datasets, and statistical test method. We trace each part of which as follows.

4.1 | Experimental scenario

We designed two different experimental scenarios for defect prediction, covering IVDP and CVDP, to explore the practicability of our WBN.

4.1.1 | In-version defect prediction

In this scenario, both training and test sets come from the same version of defect data for a project. We employed the 10-fold cross-validation method to evaluate the defect models we examined. The 10-fold cross-validation divides a dataset into 10 mutually exclusive subsets of similar size by stratified sampling technique, where 9 parts are adopted in the training phase and the remaining part is utilized in testing. To reduce the differences introduced by sample division, 10-fold cross-validation usually needs to be performed randomly and repeatedly. Therefore, in this work, we repeat the 10-fold cross-validation 100 times for each in-version experiment, and then compute the average performance of defect models.

4.1.2 | Cross-version defect prediction

In this work, we first focus on training and test data derived from different versions of defect data for the same project (Bennin et al., 2016). When considering that the two closest versions share more identical architectural and design characteristics (Xu et al., 2019), we conduct the CVDP experiment between the two nearest versions.

As shown in Figure 2, we provide the workflow of two experimental scenarios, where the target approach refers to either our chosen control method or our WBN. The biggest difference between the workflows of both scenarios is the setting of training sets, which have been described in the scenario introduction, and the rest are similar. Firstly, we established a baseline of defect prediction for the targeted approach through popular machine learners in two workflows. Then, with different validation methods, the performance of the control method and WBN are obtained in IVDP and CVDP, respectively. Finally, the nonparametric test method is used to verify the performance difference between them. The experiments have been run on a 3.6 GHz Intel Core i7-7700 machine with 16 GB RAM.

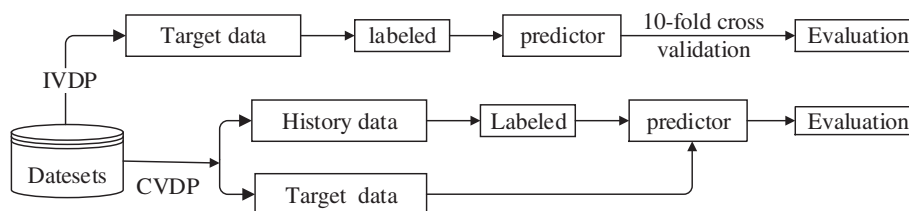


FIGURE 2 Experimental scenario settings for cross-version defect prediction (CVDP) and in-version defect prediction (IVDP)

TABLE 3 Introduction to basic information about project

Project	Version	Size	Ratio	Project	Version	Size	Ratio
Ant	1.3	125	0.160	Jedit	3.2	272	0.331
	1.4	178	0.225		4.0	306	0.245
	1.5	293	0.109		4.1	312	0.253
	1.6	351	0.262		4.2	367	0.131
	1.7	745	0.223		4.3	492	0.022
Xalan	2.4	723	0.152	Poi	1.5	237	0.595
	2.5	803	0.482		2.0	314	0.118
	2.6	885	0.464		2.5	385	0.644
	2.7	909	0.988		3.0	442	0.636
Log4j	1.0	135	0.252	Lucene	2.0	194	0.469
	1.1	109	0.339		2.2	246	0.585
	1.2	205	0.078		2.4	339	0.599
Synapse	1.0	157	0.102	Ivy	1.4	241	0.066
	1.1	222	0.270		2.0	352	0.114
	1.2	256	0.336				

4.2 | Defect datasets

According to the setting of the experimental scenario, we investigated the common and unified defect datasets in the software engineering community. To facilitate the replicability of our experiments, we designed the following rules to select the appropriate dataset:

1. the dataset is publicly available;
2. each project contains multiple versions;
3. the dataset provides bug information at file/class level;
4. the defect features of data sets are unified, that is, the dimension of features is the same.

Ultimately, we selected Jureczko datasets written in Java (Jureczko & Madeyski, 2010), also known as the PROMISE dataset in other studies. To remove diversities in defect features between versions, Ferenc et al. (2020) employed the open-source OpenStaticAnalyzer tool for analysing source code to measure uniform code indicators for each project, involving source code metrics and code duplication metrics. From the Ferenc et al. (2020) refined dataset, we selected eight open-source projects (have a total of 29 versions) that meet the experimental conditions in the improved datasets to complete this study. Their details are shown in Table 3, including the name of each project with versions, Size (i.e., the number of instances contained in the dataset, where each instance is composed of the attributes of code for a class or file), and the ratio (i.e., the probability of defect instances in the dataset). For example, there are 351 instances in ant1.6, where the defect rate is 0.262, implying that the number of defective instances in ant1.6 is $351 \times 0.262 = 92$. Also, according to this information, we can intuitively get that the class imbalance of some projects (Ant1.3, Ant1.5, Jedit4.2, Jedit4.3, Poi2.0, Log4j1.2, Synapse1.0, Ivy1.4, Ivy2.0, Xalan2.4, and Xalan2.7, Lucene) is very serious, while some projects (Xalan2.5, Xalan2.6, Poi1.5, Poi2.5, and Poi3.0) show approximate class balance.

4.3 | Learning algorithm

In this section, we describe the classifiers used to build the benchmarks and the comparison models we examined.

4.3.1 | Baseline

Four independent learners are selected from four types of learning algorithms to establish the baseline results. These algorithms include decision tree-based algorithms (i.e., J48, also known as C4.5), rule-based algorithms (i.e., JRip), instance-based algorithms (i.e., Kstar), and function-based models (i.e., SMO). The selection of these algorithms refers to previous studies, and they demonstrate the effectiveness of the algorithm in software defect prediction (Hall et al., 2012; Li, Jing, & Zhu, 2018; Wahono, 2015). When considering the implementation of the above-mentioned algorithms, we hired Weka to complete this work (Witten et al., 2011), in which the default parameters provided by Weka were used in experiments, as mentioned in many studies (Menzies et al., 2015; Tantithamthavorn et al., 2016; Zhang, Jin, et al., 2020; Y. Zhang, Xing, et al., 2020). The mean of their experimental results is chosen as a benchmark for evaluating the performance of WBN and control approaches in CVDP and IVDP, ignoring their specificity.

4.3.2 | Control approach

In defect prediction, prior studies have demonstrated that integrated learning can effectively reduce the impact of class imbalances, which have been reviewed in Section 2 (Menzies et al., 2015; Y. Zhang, Jin, et al., 2020; Y. Zhang, Xing, et al., 2020). In practice, we found that cost-sensitive learning approaches (i.e., CSForest) face a threat of generalization abilities. Therefore, four primary ensemble algorithms are employed, including Bagging, AdaBoost, Random Forest, and Vote (Galar et al., 2012; Tantithamthavorn et al., 2020). Also, we are concerned with an improved SDA method called ISDA (Jing et al., 2017), which is an approach to solve the class imbalance problem in defect prediction. The underlying classifier settings for these methods are as follows:

1. AdaBoost selects pruned C4.5 as the basic learner;
2. The base classifiers of Bagging in IVDP and CVDP are Fast decision tree and pruned C4.5 decision tree, respectively;
3. Random Forest relies on the voting choices of the decision tree to determine the final classification result;
4. Vote's combined classifiers are decision tree, logistic regression model with a ridge estimator, SVM, and Naive Bayes, and their combined strategies are majority voting;
5. We select random forest that are not sensitive to parameters as the underlying classifier for ISDA (Tantithamthavorn et al., 2016).

4.4 | Performance measurements

In this work, we select five indicators to measure the performance of machine learning, covering threshold-independent and threshold-dependent performance measures.

For threshold independent indicators, using the threshold of 0.5, we calculated two basic and two comprehensive indicators from the confusion matrix to measure the performance of machine learning (Luque et al., 2019). In practice, when performing machine learning under class imbalance data, these two basic indicators are most concerned by practitioners, including FP rate (FPR) and recall. FPR indicates how many of all negative instances are misjudged as positive (Gu et al., 2009; H. Zhang & Zhang, 2007). In other words, FPR represents the probability that a class or module without defects will be misjudged as a defect instance in the defect prediction (Khan & Ali Rana, 2019; Menzies et al., 2007). The recall is the proportion of correct positive samples predicted by the classifier to all positive samples, that is, the probability of the defect module being discovered by the defect prediction model in the defect prediction. And, F-1 and Matthews correlation coefficient (MCC) are two popular comprehensive indicators to evaluate the quality of the classification model, which can resist the interference of class imbalance. F-1 is a specific instantiation (when $\beta = 1$) of the F-measure, which is the weighted harmonic average of precision and recall. MCC is a fairly uniform index that considers true positive, true negative, false positive, and false negative (Luque et al., 2019; Yao & Shepperd, 2020). Essentially, MCC is the correlation coefficient between the actual situation and the predicted results, ranging from $[-1, 1]$, where $+1$ means perfect prediction; 0 means no better than a random prediction; and -1 means complete inconsistency between prediction and observation. Also, about threshold-dependent performance measures, we utilized AUC to measure the discriminatory power of defect models under varying the threshold, which is computed by measuring the area enclosed by the ROC curve and the coordinate axis (Flach, 2003).

For imbalanced datasets, precision and accuracy are unstable and inappropriate evaluation measures for the classifier (Menzies et al., 2007), so they were excluded in this case. In summary, FPR, recall, F-1, AUC, and MCC are selected as the criteria to evaluate the performance of the defect model we investigated.

4.5 | Statistical test

To compare and test the performance of these algorithms, past studies have exercised methods for comparative testing, for example, cross-validation *t* test, Friedman test, and Scott–Knott effect size difference test (ScottKnottESD). ScottKnottESD is selected in this paper, which is a mean comparison approach that leverages a hierarchical clustering to partition the set of treatment means into statistically distinct groups with non-negligible difference (Tantithamthavorn et al., 2016, 2017; Y. Zhang, Jin, et al., 2020). It is an alternative approach of the Scott–Knott test that considers the magnitude of the difference (i.e., effect size) of treatment means within a group and between groups (Scott & Knott, 1974). We used the *sk_esd* function implemented in the ScottKnottESD R package to compare the identification performance of the approaches we examined. Additionally, to measure the effect size of results among these algorithms and our approach, we compute the Cohen effect size estimate *d* by using the *checkDifference* function implemented in the ScottKnottESD R package. The magnitude is accessed using the thresholds as follows: negligible ($d \leq 0.2$), small ($0.2 < d \leq 0.5$), medium ($0.5 < d \leq 0.8$), and large ($d > 0.8$). Finally, according to the test results, the performance rankings of all the algorithms on all the experimental objects are obtained.

5 | ANALYSIS OF EXPERIMENTAL RESULTS

In this section, according to different scenarios (i.e., IVDP and CVDP), the results of defect prediction are presented, including baseline, five control approaches, and WBN, where the baseline is constructed from the results of four independent learners. Moreover, the results of the different tests for these algorithms compared with the baseline are given.

5.1 | IVDP results

Table 4 shows that the average results of four machine learning in CVDP, which are obtained by running 100 times 10-fold cross-validation for IVDP in 26 experiments from 9 projects. The FPR, recall, F-1, MCC, and AUC for each project(or version) are shown by row.

From Table 4, we found that the average results of each index are 0.501, 0.817, 0.806, 0.346, 0.679, respectively, which are baselines for evaluating WBN in IVDP. Besides, to clarify the impact of class imbalance on different defect prediction indicators, we performed a correlation analysis between the class ratio and five performance indicators. The experimental results show that FPR, MCC, and AUC have a significantly strong correlation with the class ratio at 0.001 confidence level, with Pearson correlation coefficients of -0.703 , 0.754 , and 0.806 , respectively. Namely, defect models on data with serious class imbalances yield higher FPR than those with similar balances, indicating that the class imbalance of the dataset hurts FPR in IVDP. For example, the FPR of Jedit4.3 and Log4j1.2 are 0.849 and 0.850, respectively, which are 0.604 and 0.605 higher than that of Poi2.5, whose value is 0.245. AUC and MCC performed similarly to FPR, but recall and F-1 did not. In IVDP, this information provides a crucial reference for analysing five comparison methods and our WBN.

Table 5 presents the IVDP results of approaches we examined in 26 experiments, covering five contrast approaches and our WBN. The table is divided into six parts according to different algorithms, each of which is similar to Table 4. Beyond that, the last two lines of each part provide the mean value of the specific algorithm in all cases and the Cohen effect size estimated *d* (called S.K.'diff), in which N denotes negligible, S denotes small, M denotes medium, and L denotes large.

When considering the mean, the performance of these algorithms is better than that of baseline. For FPR, ISDA is the lowest of all algorithms, with an average value of 0.241, followed by WBN, which has a mean of 0.384. Compared with ISDA, our WBN not only reduced the FPR but maintained recall whose mean value was 0.797, whereas ISDA did not, with an average recall of only 0.397. The average MCC for WBN is lower and higher compared with the RandomForest and the remaining contrast methods, respectively, where WBN's MCC surpasses RandomForest in 11 of 25 projects. When average AUC is considered, although our method is worse than other algorithms (RandomForest, Bagging, and Adaboost), it exceeds the ISDA and Vote. From Table 5, the AUC of WBN is better than that of ISDA in all experiments. The recall and F-1 of our WBN reveal generality compared with other defect models other than ISDA.

Combined with the defect prediction results in Table 5, we observe that the specific algorithms have great differences under different projects. Our WBN obtained lower FPR in Poi3.0 and Log4j1.1 than other experiments, and its values were 0.193 and 0.197, respectively. In other experiments, our WBN has a similar performance, which shows that adjusting the sampled weight of the instance through the classification error rate can effectively reduce the false alarm of the defect model in IVDP. For example, by comparing the best control method, that is, random forest, our WBN achieved lower FPR in 19 of 26 experiments. According to Cohen's *d* and its magnitude for the effect size, we can intuitively understand that the Cohen's *d* of FPR between WBN and baseline is 0.792 and its magnitude is M, namely, the WBN outperforms baseline when considering the FPR in 19 out of 26 projects. Except for ISDA, the performance of other control methods under FPR is not significantly different from baseline experiments, also illustrating that our method effectively reduces false alarms and maintains the recall in IVDP. AUC and MCC are also indicators greatly affected by class imbalance in IVDP. They behaved similarly to recall and F-1, showing generality in these experiments.

TABLE 4 The average of evaluation results for four independent learners on in-version defect prediction

	FPR	Recall	F-1	MCC	AUC
ant1.3	0.599	0.829	0.813	0.272	0.640
ant1.4	0.615	0.751	0.714	0.135	0.575
ant1.5	0.599	0.829	0.813	0.272	0.640
ant1.6	0.381	0.785	0.779	0.423	0.720
ant1.7	0.452	0.810	0.798	0.406	0.705
jedit3.2	0.368	0.746	0.733	0.406	0.708
jedit4.0	0.435	0.799	0.784	0.409	0.688
jedit4.1	0.395	0.802	0.793	0.452	0.735
jedit4.2	0.566	0.863	0.853	0.341	0.662
jedit4.3	0.849	0.971	0.966	0.118	0.603
poi1.5	0.326	0.695	0.694	0.373	0.702
poi2.0	0.767	0.861	0.836	0.140	0.586
poi2.5	0.245	0.789	0.789	0.541	0.798
poi3.0	0.259	0.769	0.770	0.506	0.776
log4j1.0	0.345	0.832	0.824	0.530	0.731
log4j1.1	0.318	0.782	0.775	0.501	0.737
log4j1.2	0.850	0.896	0.878	0.073	0.541
synapse1.0	0.797	0.862	0.843	0.063	0.549
synapse1.1	0.406	0.774	0.766	0.409	0.707
synapse1.2	0.374	0.734	0.726	0.386	0.692
ivy1.4	0.772	0.926	0.912	0.208	0.586
ivy2.0	0.644	0.872	0.860	0.272	0.642
xalan2.4	0.653	0.832	0.811	0.229	0.622
xalan2.5	0.296	0.706	0.705	0.411	0.728
xalan2.6	0.265	0.747	0.743	0.493	0.761
xalan2.7	0.445	0.991	0.991	0.620	0.811
Avg.	0.501	0.817	0.806	0.346	0.679

Abbreviations: AUC, area under the ROC curve; FPR, false-positive rate; MCC, Matthews correlation coefficient.

Through the above information, we can know more about the performance of WBN, but we cannot determine whether it is better or worse than other algorithms. Through the ScottKnottESD test, Figure 3 provides the ranking of various algorithms under FPR, MCC, and AUC in the form of first, second, ..., until the end. Our algorithm WBN ranked second both after ISDA under FPR and after random forest under MCC, respectively. And, under AUC, our WBN is centred on the performance ranking among all the methods we examined. In summary, our approach has more or less improved in IVDP, most notably FPR.

5.2 | CVDP results

Table 6 presents the average results of four independent learners for CVDP in 21 experiments, with FPR, recall, F-1, MCC, and AUC for each object by row. From Table 3, we can clearly observe the difference of class imbalance between the training set and the test set for CVDP experiments. Like IVDP, we executed a correlation analysis between five performance indicators and class ratio difference between the training set and test set. The experimental results show that there is no significant correlation between them. And then, according to the ratio of class imbalance of the training set to that of the test set, we divided the experiments into several categories:

1. imbalance–imbalance (abbr. to imb.–imb.) where the class ratio of the training set and test set is less than 40% or greater than 60%, including four experiments from Ant, four experiments from Jedit, two experiments from Log4j, poi2.5–3.0, two experiments from Synapse, and one experiment from Ivy, that is, ivy1.4–2.0;

TABLE 5 Experimental results of five comparison methods and our WBN on in-version defect prediction

	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC
	Bagging					Adaboost					ISDA				
ant1.3	0.638	0.840	0.817	0.273	0.733	0.608	0.784	0.782	0.180	0.700	0.096	0.200	0.235	0.121	0.552
ant1.4	0.689	0.766	0.712	0.130	0.713	0.556	0.748	0.735	0.213	0.684	0.261	0.100	0.100	-0.161	0.420
ant1.5	0.749	0.895	0.868	0.265	0.743	0.604	0.897	0.885	0.372	0.726	0.154	0.125	0.105	-0.025	0.486
ant1.6	0.336	0.808	0.805	0.489	0.837	0.382	0.780	0.776	0.414	0.813	0.115	0.500	0.548	0.411	0.692
ant1.7	0.421	0.831	0.820	0.467	0.826	0.427	0.805	0.799	0.405	0.794	0.134	0.277	0.317	0.159	0.571
jedit3.2	0.311	0.779	0.775	0.487	0.816	0.324	0.754	0.752	0.436	0.796	0.253	0.378	0.400	0.129	0.563
jedit4.0	0.382	0.824	0.815	0.490	0.797	0.376	0.804	0.799	0.449	0.783	0.164	0.289	0.324	0.137	0.563
jedit4.1	0.342	0.834	0.827	0.535	0.836	0.370	0.808	0.802	0.466	0.822	0.207	0.525	0.494	0.307	0.659
jedit4.2	0.611	0.883	0.865	0.370	0.820	0.549	0.872	0.863	0.371	0.803	0.113	0.333	0.320	0.214	0.610
jedit4.3	0.921	0.978	0.969	0.165	0.703	0.710	0.976	0.973	0.332	0.681	0.000	0.000	0.000	0.000	0.500
poi1.5	0.311	0.721	0.719	0.416	0.784	0.338	0.684	0.684	0.345	0.766	0.479	0.671	0.671	0.192	0.596
poi2.0	0.849	0.878	0.833	0.080	0.721	0.731	0.864	0.844	0.180	0.712	0.058	0.111	0.143	0.069	0.527
poi2.5	0.226	0.807	0.807	0.580	0.887	0.232	0.808	0.807	0.579	0.879	0.441	0.694	0.717	0.247	0.626
poi3.0	0.247	0.784	0.784	0.535	0.858	0.268	0.772	0.772	0.507	0.845	0.400	0.700	0.726	0.294	0.650
log4j1.0	0.273	0.859	0.856	0.612	0.854	0.339	0.805	0.803	0.475	0.818	0.080	0.529	0.600	0.495	0.725
log4j1.1	0.283	0.813	0.807	0.570	0.823	0.280	0.774	0.773	0.495	0.812	0.083	0.278	0.385	0.258	0.597
log4j1.2	0.905	0.919	0.886	0.045	0.643	0.702	0.913	0.902	0.269	0.733	0.750	0.883	0.907	0.107	0.566
synapse1.0	0.885	0.891	0.849	0.017	0.706	0.752	0.867	0.854	0.142	0.674	0.100	0.125	0.125	0.025	0.513
synapse1.1	0.439	0.791	0.773	0.417	0.768	0.367	0.784	0.780	0.435	0.794	0.370	0.433	0.356	0.057	0.531
synapse1.2	0.353	0.757	0.749	0.432	0.770	0.358	0.728	0.725	0.379	0.756	0.341	0.535	0.484	0.186	0.597
ivy1.4	0.891	0.933	0.906	0.125	0.806	0.737	0.935	0.921	0.305	0.828	0.098	0.375	0.273	0.215	0.638
ivy2.0	0.732	0.884	0.860	0.238	0.797	0.647	0.872	0.861	0.266	0.791	0.071	0.250	0.278	0.198	0.590
xalan2.4	0.676	0.849	0.821	0.255	0.803	0.604	0.837	0.823	0.277	0.774	0.101	0.291	0.314	0.202	0.595
xalan2.5	0.250	0.749	0.749	0.498	0.823	0.259	0.741	0.741	0.482	0.814	0.106	0.163	0.255	0.084	0.529
xalan2.6	0.232	0.773	0.772	0.542	0.860	0.246	0.757	0.757	0.512	0.822	0.295	0.573	0.599	0.280	0.639
xalan2.7	0.538	0.993	0.992	0.662	0.866	0.448	0.992	0.992	0.654	0.790	1.000	0.987	0.987	-0.013	0.493
Avg.	0.519	0.840	0.824	0.373	0.792	0.470	0.822	0.816	0.382	0.777	0.241	0.397	0.410	0.161	0.578
S.K' diff	0.081,N	0.315,S	0.255,S	0.157,N	1.323,L	0.169,N	0.056,N	0.123,N	0.258,S	1.089,L	1.222,L	2.266,L	2.138,L	1.246,L	1.482,L
	Random Forest					Vote					WBN				
ant1.3	0.759	0.840	0.792	0.169	0.773	0.524	0.800	0.802	0.271	0.638	0.329	0.760	0.784	0.349	0.768
ant1.4	0.640	0.773	0.732	0.194	0.710	0.492	0.733	0.734	0.239	0.620	0.653	0.708	0.688	0.063	0.587
ant1.5	0.630	0.853	0.827	0.320	0.792	0.532	0.897	0.890	0.412	0.682	0.315	0.832	0.853	0.400	0.769

TABLE 5 (Continued)

	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC
ant1.6	0.317	0.813	0.811	0.508	0.853	0.412	0.793	0.782	0.425	0.690	0.223	0.807	0.813	0.547	0.819
ant1.7	0.397	0.842	0.832	0.503	0.847	0.443	0.832	0.818	0.461	0.694	0.288	0.747	0.763	0.403	0.788
jedit3.2	0.304	0.772	0.770	0.477	0.847	0.320	0.761	0.758	0.450	0.720	0.265	0.713	0.721	0.423	0.798
jedit4.0	0.360	0.828	0.821	0.507	0.826	0.422	0.828	0.812	0.487	0.703	0.322	0.798	0.800	0.465	0.782
jedit4.1	0.347	0.835	0.827	0.536	0.864	0.383	0.829	0.818	0.511	0.723	0.283	0.802	0.806	0.500	0.815
jedit4.2	0.577	0.885	0.871	0.399	0.836	0.635	0.878	0.859	0.340	0.622	0.346	0.813	0.832	0.381	0.813
jedit4.3	0.711	0.979	0.975	0.387	0.867	0.720	0.974	0.971	0.299	0.627	0.628	0.979	0.976	0.431	0.724
poi1.5	0.283	0.742	0.741	0.462	0.809	0.281	0.726	0.727	0.440	0.722	0.319	0.691	0.693	0.369	0.739
poi2.0	0.783	0.884	0.849	0.204	0.792	0.753	0.876	0.849	0.198	0.561	0.551	0.771	0.796	0.176	0.683
poi2.5	0.197	0.835	0.835	0.639	0.912	0.223	0.800	0.801	0.570	0.789	0.207	0.808	0.809	0.591	0.845
poi3.0	0.232	0.801	0.801	0.570	0.881	0.239	0.767	0.770	0.516	0.764	0.193	0.788	0.791	0.575	0.849
log4j1.0	0.308	0.833	0.830	0.544	0.858	0.355	0.816	0.811	0.489	0.731	0.197	0.837	0.842	0.603	0.849
log4j1.1	0.275	0.804	0.800	0.551	0.831	0.292	0.793	0.788	0.525	0.750	0.270	0.806	0.803	0.556	0.792
log4j1.2	0.816	0.926	0.901	0.249	0.676	0.774	0.869	0.869	0.094	0.547	0.862	0.898	0.879	0.053	0.627
synapse1.0	0.850	0.883	0.851	0.058	0.746	0.779	0.851	0.841	0.081	0.536	0.693	0.852	0.849	0.164	0.691
synapse1.1	0.438	0.785	0.769	0.403	0.798	0.413	0.803	0.788	0.455	0.695	0.417	0.731	0.730	0.315	0.739
synapse1.2	0.309	0.787	0.781	0.506	0.793	0.356	0.774	0.762	0.467	0.709	0.316	0.736	0.737	0.415	0.738
ivy1.4	0.888	0.931	0.906	0.104	0.856	0.730	0.921	0.913	0.239	0.596	0.691	0.862	0.876	0.136	0.651
ivy2.0	0.647	0.891	0.875	0.333	0.835	0.553	0.889	0.882	0.385	0.668	0.291	0.829	0.850	0.416	0.821
xalan2.4	0.636	0.857	0.833	0.314	0.821	0.695	0.848	0.817	0.240	0.577	0.289	0.745	0.776	0.352	0.795
xalan2.5	0.221	0.777	0.777	0.556	0.857	0.310	0.697	0.694	0.396	0.694	0.346	0.655	0.655	0.309	0.722
xalan2.6	0.218	0.785	0.785	0.568	0.874	0.283	0.740	0.732	0.488	0.728	0.241	0.766	0.765	0.528	0.840
xalan2.7	0.449	0.994	0.994	0.737	0.883	0.449	0.988	0.988	0.513	0.769	0.449	0.993	0.993	0.666	0.778
Avg.	0.484	0.844	0.830	0.415	0.824	0.476	0.826	0.818	0.384	0.675	0.384	0.797	0.803	0.392	0.762
S.K.'diff	0.079,N	0.379,S	0.346,S	0.425,S	1.323,L	0.135,N	0.121,N	0.162,N	0.264,S	1.089,L	0.623,M	0.263,S	0.044,N	0.287,S	1.482,L

Abbreviations: AUC, area under the ROC curve; FPR, false-positive rate; MCC, Matthews correlation coefficient; WBN, BayesNet with adjustable weights.

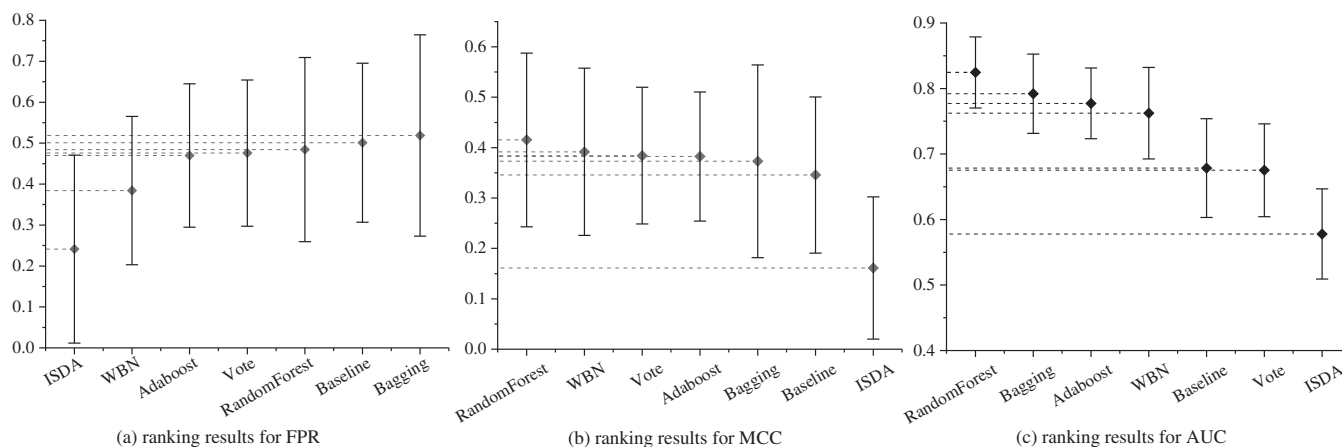


FIGURE 3 Performance ranking of our BayesNet with adjustable weights (WBN) and five control methods compared to the baseline in in-version defect prediction (IVDP)

TABLE 6 The average of evaluation results for four independent learners on cross-version defect prediction

	FPR	Recall	F-1	MCC	AUC
ant1.3–1.4	0.688	0.723	0.687	0.042	0.546
ant-1.4–1.5	0.714	0.767	0.787	0.040	0.592
ant-1.5–1.6	0.628	0.752	0.693	0.221	0.591
ant1.6–1.7	0.421	0.781	0.779	0.368	0.688
jedit3.2–4.0	0.323	0.790	0.792	0.457	0.746
jedit4.0–4.1	0.381	0.812	0.802	0.482	0.713
jedit4.1–4.2	0.345	0.824	0.840	0.407	0.760
jedit4.2–4.3	0.579	0.925	0.944	0.214	0.698
log4j1.0–1.1	0.299	0.796	0.789	0.533	0.755
log4j1.1–1.2	0.216	0.306	0.389	0.057	0.539
lucene2.0–2.2	0.357	0.598	0.595	0.245	0.620
lucene2.2–2.4	0.406	0.637	0.630	0.240	0.631
poi1.5–2.0	0.399	0.379	0.458	−0.014	0.481
poi2.0–2.5	0.368	0.381	0.266	0.037	0.510
poi2.5–3.0	0.378	0.669	0.669	0.291	0.652
synapse1.0–1.1	0.652	0.704	0.652	0.078	0.515
synapse1.1–1.2	0.503	0.686	0.655	0.229	0.581
xalan2.4–2.5	0.483	0.547	0.452	0.114	0.531
xalan2.5–2.6	0.345	0.652	0.653	0.307	0.666
xalan2.6–2.7	0.030	0.419	0.574	0.087	0.665
ivy1.4–2.0	0.750	0.875	0.851	0.181	0.657
imb.-imb.	0.491	0.744	0.738	0.257	0.645
imb.-b.	0.320	0.432	0.437	0.056	0.547
b.-b.	0.369	0.629	0.626	0.264	0.639
Avg.	0.441	0.668	0.665	0.220	0.625

Abbreviations: AUC, area under the ROC curve; FPR, false-positive rate; MCC, Matthews correlation coefficient.

- balance-imbalance or imbalance-balance (abbr. to imb.-b.) in which when the value range of the class ratio of the training/test set is 40%–60%, the value of the class ratio of the test/training set takes values in other ranges, covering xalan2.4–2.5, xalan2.6–2.7, poi1.5–poi2.0, and poi2.0–poi2.5;

3. balance–balance (abbr. to b.–b.) where the range of class ratio values for both the training and test sets is 40%–60%, involving xalan2.5–2.6 and two experiments from Lucene.

We can observe that the average results of each indicator are 0.441, 0.668, 0.665, 0.220, 0.625, respectively, which are baselines for evaluating the performance of the algorithms we examined in CVDP. For the experiments of imb.–imb., their mean FPR was not only higher than the baseline we set, but also higher than other groups. Although its FPR (0.491) is worse than other groups, other indicators showed better performance, including recall, F-1, MCC, and AUC, whose values were 0.744, 0.738, 0.257, and 0.645. Relatively, the experiments of b.–b. is ordinary, or more correctly, their performance are higher and lower than the imb.–b. and the imb.–imb., respectively. Since the class distributions of the training and test sets violate the machine-learned assumptions, the results of experiments for imb.–b. performed worst in all experiments, no matter which index. For example, in poi1.5–2.0, recall and MCC are 0.379 and –0.014, respectively, and the remaining experiments from imb.–b. are similar to it. In CVDP, the results of baseline provide an important reference for analysing five control methods and our WBN.

The design of Table 7 is the same as Table 5, giving the results of six defect models in 21 CVDP experiments from eight projects. The best-performance values for WBN have been highlighted in Table 7. Compared with baseline and these five control methods, WBN is the most efficient in CVDP for reducing false alarms and improving reliability. When considering the mean of the performance measures, our WBN achieved the best performance under four measures (i.e., FPR, F-1, MCC, and AUC), with means of 0.322, 0.697, 0.302, and 0.734, respectively. In different groups, so did our WBN under these four indicators. The recall of WBN is second only to that of random forest, with a value of 0.674. According to different groups, we found that although the mean recall of our WBN in 14 experiments from imb.–imb. is worse than that of random forest, the mean recall of WBN is better than that of random forest in the remaining experiments from other groups.

The imb.–imb. is the most common pattern in CVDP, which is the pattern matching the training set and the test set. By observing Table 7, we can clearly find that our approach WBN reduced the FPR against baseline by an average of 33.2% in this pattern. In 12 out of 14 experiments from imb.–imb., our WBN obtained the lowest false alarm compared with the Vote, in which the average FPR is reduced by 26.1%. In 8 out of 14 experiments from imb.–imb., our WBN performed better than the best control method Vote under MCC, with an average MCC increase of 7.3%. Especially in experiments ant1.4–1.5 and synapse1.1–1.2, compared with the Vote, our method improves 81.2% and 56.2% under MCC, respectively. In imb.–imb., AUC is the same as FPR and MCC, and recall and F-1 show commonality compared with the comparison method.

Since the b.–b. meets the preconditions of machine learning, it is an ideal model in CVDP. The three experiments from the b.–b. show that our method WBN achieves encouraging CVDP performance against five control methods and baseline we designed, with means of 0.330 and 0.734 for the MCC and AUC, respectively. While, for CVDP, the imb.–b. is a poor pattern versus the imb.–imb. and the b.–b. The four experiments from the imb.–b. show that all the defect models we examined, including WBN and baseline, have poor performance reliability, especially for recall, with our WBN and AdaBoost performing the best against the others.

Overall analysis revealed that, as shown in Table 7, against the other five defect predictors, WBN achieved the best F-1, the best MCC, and the best AUC in 6, 8, and 10 experiments of 21 experiments, respectively. According to the results of statistical tests, we found that the performance of two control methods (i.e., ISDA and Vote) under FPR is significantly different from baseline experiments. The Cohen's *d* of FPR between WBN and baseline is 0.799 and its magnitude is M, which is higher and lower than ISDA and Vote respectively. Namely, the WBN has a lower FPR than baseline when considering the FPR in 19 out of 21 projects. About recall, there was no significant difference between all methods we inspected (including WBN) and baseline methods. And, the Cohen's *d* of F-1 between WBN and baseline is 0.220 and its magnitude is S, while other comparison methods (except ISDA) have no significant difference from the baseline. Under AUC and MCC, the Cohen's *d* between WBN and baseline are 0.506 and 1.125, respectively, and their effect sizes are M and L, respectively. These test results reveal that our WBN is significantly better than the baseline and five comparison methods, which can effectively reduce the false positives in CVDP and maintain the accuracy of defect prediction.

Figure 4 provides the performance ranking of six approaches, covering five control approaches and WBN, against baseline under FPR, MCC, and AUC through the ScottKnottESD test in CVDP. FPR for WBN ranks second only to ISDA. Unlike ISDA, our WBN maintained recall in CVDP when reducing FPR. As shown in Table 7, the recall of our WBN is 0.674, while the ISDA is only 0.407. Therefore, combined with recall, our method is better than ISDA. WBN is obviously superior to other algorithms under MCC and AUC in CVDP, where the ranking for WBN is first, showing the effectiveness of our WBN for reducing the false positives in defect prediction.

To summary, our WBN is superior to the five control approaches and baseline in IVDP and CVDP. Although both ISDA and WBN are customized methods for solving class imbalance problems in defect prediction, the principles of the two methods are different. The method principle of ISDA is to achieve balanced subclasses through subclass discriminant analysis, and then learn the defect model on this balanced data subset to predict the potential defects in the target domain. In other words, this method changes the difference of class imbalance between the training set and test set in defect prediction, so that the matching pattern between the training set and test set fell into the imb.–b., which violates the preconditions of machine learning. Our analysis in CVDP shows that the imb.–b. model is not expected in defect prediction. From our analysis in CVDP, we found that the imb.–b. model is not expected in defect prediction because the reliability of the defect model obtained under this mode is lower than the other two patterns. The mechanism of our WBN is to change the sampling probability of the misclassified instances when training the defect model for CVDP. In this process, our method does not change the difference of class imbalance between the source domain and

TABLE 7 Experimental results of five control approaches and our WBN on cross-version defect prediction

	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC
	Bagging					AdaBoost					ISDA				
ant1.3-1.4	0.714	0.742	0.690	0.043	0.588	0.687	0.713	0.683	0.033	0.658	0.130	0.200	0.242	0.082	0.535
ant1.4-1.5	0.814	0.850	0.831	0.047	0.642	0.768	0.775	0.790	0.006	0.542	0.146	0.156	0.133	0.009	0.505
ant1.5-1.6	0.706	0.749	0.653	0.180	0.731	0.647	0.738	0.679	0.153	0.708	0.066	0.207	0.297	0.204	0.570
ant1.6-1.7	0.422	0.801	0.796	0.400	0.793	0.475	0.783	0.774	0.331	0.745	0.206	0.512	0.459	0.286	0.653
jedit3.2-4.0	0.318	0.768	0.775	0.422	0.818	0.300	0.768	0.777	0.433	0.791	0.186	0.507	0.487	0.313	0.660
jedit4.0-4.1	0.364	0.814	0.808	0.482	0.806	0.357	0.811	0.806	0.478	0.813	0.099	0.443	0.511	0.385	0.672
jedit4.1-4.2	0.309	0.826	0.843	0.422	0.841	0.262	0.785	0.813	0.394	0.818	0.179	0.604	0.433	0.339	0.713
jedit4.2-4.3	0.535	0.931	0.947	0.234	0.717	0.624	0.892	0.924	0.131	0.688	0.100	0.273	0.097	0.084	0.586
log4j1.0-1.1	0.239	0.817	0.815	0.586	0.834	0.257	0.807	0.805	0.563	0.841	0.097	0.622	0.687	0.556	0.762
log4j1.1-1.2	0.229	0.327	0.417	0.059	0.599	0.227	0.351	0.446	0.073	0.558	0.125	0.423	0.590	0.163	0.649
lucene2.0-2.2	0.408	0.565	0.566	0.156	0.617	0.342	0.630	0.632	0.286	0.657	0.275	0.479	0.573	0.206	0.602
lucene2.2-2.4	0.402	0.614	0.616	0.209	0.673	0.416	0.611	0.612	0.194	0.634	0.463	0.591	0.622	0.126	0.564
poi1.5-2.0	0.387	0.385	0.466	-0.001	0.535	0.336	0.411	0.491	0.050	0.546	0.592	0.784	0.252	0.127	0.596
poi2.0-2.5	0.355	0.392	0.282	0.070	0.416	0.362	0.410	0.329	0.074	0.421	0.124	0.133	0.221	0.013	0.504
poi2.5-3.0	0.350	0.672	0.676	0.315	0.689	0.343	0.674	0.678	0.322	0.712	0.422	0.651	0.688	0.222	0.614
synapse1.0-1.1	0.665	0.734	0.660	0.141	0.611	0.696	0.707	0.631	0.021	0.623	0.093	0.200	0.276	0.146	0.554
synapse1.1-1.2	0.503	0.699	0.664	0.253	0.646	0.480	0.676	0.658	0.220	0.677	0.147	0.326	0.403	0.208	0.589
xalan2.4-2.5	0.479	0.549	0.466	0.113	0.584	0.458	0.569	0.498	0.168	0.561	0.106	0.163	0.255	0.084	0.529
xalan2.5-2.6	0.350	0.649	0.649	0.298	0.683	0.321	0.675	0.675	0.353	0.673	0.462	0.667	0.606	0.205	0.602
xalan2.6-2.7	0.007	0.442	0.600	0.096	0.935	0.007	0.455	0.613	0.099	0.719	0.000	0.478	0.647	0.105	0.739
ivy1.4-2.0	0.800	0.884	0.849	0.169	0.785	0.800	0.881	0.848	0.154	0.667	0.054	0.125	0.161	0.092	0.535
imb.-imb.	0.498	0.758	0.745	0.268	0.721	0.495	0.740	0.737	0.237	0.703	0.146	0.375	0.390	0.221	0.614
imb.-b.	0.307	0.442	0.454	0.070	0.618	0.291	0.461	0.483	0.098	0.562	0.205	0.389	0.344	0.082	0.592
b.-b.	0.387	0.609	0.610	0.221	0.658	0.360	0.639	0.640	0.278	0.655	0.400	0.579	0.600	0.179	0.590
Avg.	0.446	0.677	0.670	0.224	0.693	0.436	0.672	0.674	0.216	0.669	0.194	0.407	0.411	0.188	0.606
S.K.'diff	0.023,N	0.051,N	0.033,N	0.023,N	0.636,M	0.024,N	0.028,N	0.063,N	0.023,N	0.458,S	1.471,L	1.372,L	1.397,L	0.213,S	0.239,S
	Random Forest					Vote					WBN				
ant1.3-1.4	0.680	0.736	0.698	0.075	0.650	0.611	0.730	0.712	0.137	0.560	0.542	0.663	0.675	0.113	0.594
ant1.4-1.5	0.873	0.812	0.802	-0.068	0.641	0.468	0.765	0.799	0.218	0.648	0.408	0.802	0.828	0.301	0.746
ant1.5-1.6	0.628	0.752	0.695	0.213	0.752	0.590	0.761	0.715	0.261	0.586	0.350	0.803	0.799	0.473	0.816
ant1.6-1.7	0.407	0.811	0.806	0.427	0.820	0.446	0.807	0.797	0.397	0.680	0.286	0.750	0.766	0.408	0.803
jedit3.2-4.0	0.270	0.807	0.812	0.511	0.838	0.246	0.742	0.757	0.438	0.748	0.253	0.693	0.714	0.381	0.796

TABLE 7 (Continued)

	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC	FPR	Recall	F-1	MCC	AUC
jedit4.0-4.1	0.353	0.824	0.817	0.507	0.847	0.371	0.843	0.831	0.550	0.736	0.246	0.817	0.822	0.545	0.819
jedit4.1-4.2	0.274	0.826	0.844	0.443	0.850	0.305	0.856	0.866	0.475	0.775	0.195	0.760	0.796	0.408	0.864
jedit4.2-4.3	0.624	0.925	0.943	0.176	0.633	0.623	0.937	0.950	0.200	0.657	0.449	0.766	0.849	0.110	0.668
log4j1.0-1.1	0.239	0.817	0.815	0.586	0.843	0.233	0.853	0.848	0.665	0.810	0.248	0.798	0.798	0.550	0.845
log4j1.1-1.2	0.228	0.337	0.429	0.064	0.595	0.224	0.380	0.480	0.089	0.578	0.167	0.376	0.471	0.120	0.650
lucene2.0-2.2	0.365	0.593	0.592	0.230	0.688	0.337	0.602	0.594	0.275	0.633	0.342	0.618	0.617	0.277	0.679
lucene2.2-2.4	0.408	0.619	0.620	0.211	0.677	0.324	0.658	0.661	0.327	0.667	0.283	0.661	0.660	0.379	0.755
poi1.5-2.0	0.364	0.379	0.457	0.010	0.522	0.454	0.583	0.657	0.084	0.564	0.404	0.430	0.514	0.017	0.548
poi2.0-2.5	0.363	0.403	0.313	0.064	0.408	0.368	0.369	0.238	0.001	0.500	0.397	0.442	0.411	0.051	0.538
poi2.5-3.0	0.339	0.686	0.689	0.339	0.732	0.317	0.710	0.713	0.387	0.697	0.304	0.724	0.726	0.414	0.826
synapse1.0-1.1	0.679	0.725	0.648	0.095	0.654	0.588	0.743	0.699	0.226	0.577	0.429	0.721	0.721	0.292	0.649
synapse1.1-1.2	0.487	0.695	0.667	0.252	0.734	0.499	0.707	0.670	0.274	0.604	0.349	0.754	0.747	0.428	0.703
xalan2.4-2.5	0.471	0.558	0.475	0.143	0.616	0.495	0.538	0.410	0.112	0.521	0.395	0.618	0.606	0.238	0.630
xalan2.5-2.6	0.307	0.688	0.688	0.381	0.708	0.337	0.673	0.672	0.341	0.668	0.332	0.667	0.667	0.334	0.746
xalan2.6-2.7	0.007	0.454	0.611	0.099	0.935	0.009	0.305	0.452	0.071	0.648	0.007	0.436	0.593	0.095	0.912
ivy1.4-2.0	0.822	0.881	0.844	0.126	0.800	0.690	0.889	0.869	0.295	0.600	0.368	0.847	0.860	0.398	0.820
imb.-imb.	0.493	0.760	0.751	0.268	0.742	0.444	0.766	0.765	0.329	0.661	0.328	0.734	0.755	0.353	0.757
imb.-b.	0.301	0.449	0.464	0.079	0.620	0.332	0.449	0.439	0.067	0.558	0.301	0.482	0.531	0.100	0.657
b.-b.	0.360	0.633	0.633	0.274	0.691	0.333	0.644	0.642	0.314	0.656	0.319	0.649	0.648	0.330	0.727
Avg.	0.438	0.682	0.679	0.233	0.712	0.406	0.688	0.685	0.277	0.641	0.322	0.674	0.697	0.302	0.734
S.K.' diff	0.018,N	0.084,N	0.090,N	0.074,N	0.805,L	0.202,S	0.117,N	0.122,N	0.348,S	0.184,N	0.799,M	0.037,N	0.220,S	0.506,M	1.125,L

Abbreviations: AUC, area under the ROC curve; FPR, false-positive rate; MCC, Matthews correlation coefficient; WBN, BayesNet with adjustable weights.

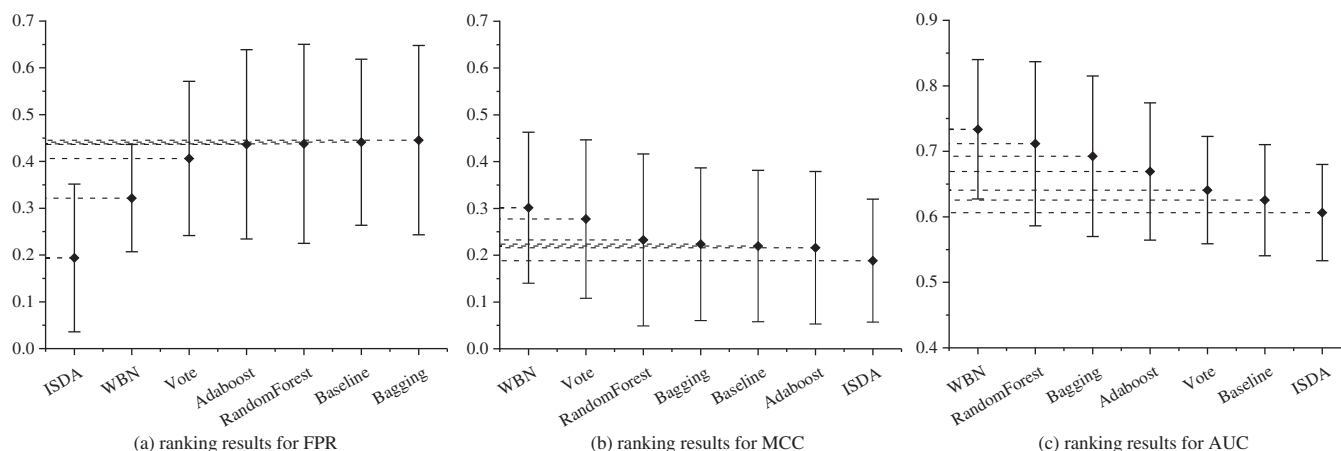


FIGURE 4 Performance ranking of our BayesNet with adjustable weights (WBN) and five control methods compared to the baseline in cross-version defect prediction (CVDP)

the target domain, while it increases the learning opportunities for misclassified instances, enabling defect models to more thoroughly learn the characteristics of potential defective modules/classes.

6 | DISCUSSION

In this section, we discuss the practical implications of our approach on the software engineering community.

In defect prediction, the defect model learning under imbalanced data can produce false alarms, which is inevitable (Arar & Ayan, 2015). False alarms have to pay a price for confirming whether it is a defect in the software life cycle. Anecdotal evidence suggests that the costs of defects depend strongly on the project context and themselves, including location, size, or something. Some defects can be expensive, while others can be cheap (Herbold, 2019). Besides, their costs are closely related to application fields, including aerospace applications, high-precision medical equipment, and Internet applications in the financial (H. Gao et al., 2021). According to the experience of software development and testing, software testers and developers need to manually check and confirm the results of defect prediction, where manual code inspections are quite labour-intensive (X. Ma et al., 2021; Shull et al., 2002a). Depending on the review methods, 8–20 lines of code can be inspected per minute per reviewer, where a review team consists of four or six members (Bener et al., 2015; Shull et al., 2002b). Therefore, the false alarm can increase the workload of testing and development, consuming unnecessary costs.

Although FPR has been ignored, it is a necessary and irreplaceable indicator. In this paper, our method WBN effectively reduces false positives, with an average FPR of 0.384 and 0.322 in IVDP and CVDP, respectively. In this research, we found that the difference of class imbalance between the training set and the test set harms the performance of CVDP, which has a greater impact on recall than FPR. At present, according to the practical experience of the software engineering community, we set the defect data of the adjacent version for the target version as the training set (i.e., the defect data of the latest historical version). When considering the uncertainty of defect distribution, in the same project, the learning defect prediction model on training sets from different historical versions has diverse reliability for predicting potential defects in the target version. We selected three target versions ant1.6, jedit4.2, and poi3.0 from Ant, Jedit, and Poi, respectively. And then, we performed three CVDP experiments for each target version using random forest on their defect data of the different historical versions, where their MCC results are shown in Figure 5. The results reveal that the accuracy of defect prediction varies for the target versions using defect data of the different historical versions. For example, the training set poi1.5 to the poi3.0, their MCC is the best compared with other combinations (i.e., poi2.0–poi3.0 and poi2.5–poi3.0), with its value of 0.35. Similar behaviour was observed in the other two experiments. Based on this information, it is recommended that practitioners select an appropriate training set from historical versions to predict the defects of the unpublished version.

How to reduce the FPR of classifiers, guaranteeing its accuracy, is a difficult problem. In addition to our proposed WBN, several other approaches have been proposed to provide reference evidence for practitioners in related areas. WBN is not conflicting with the rebalancing technique, that is, WBN can be used together with class rebalancing technique to predict potential defect modules in the target entity. According to the performance of WBN in the two defect prediction scenarios, we consider recommending WBN to maintain the reliability of the target version using historical versions of defect data. Also, WBN should be employed to reduce unnecessary cost waste when resources for development and testing are limited.

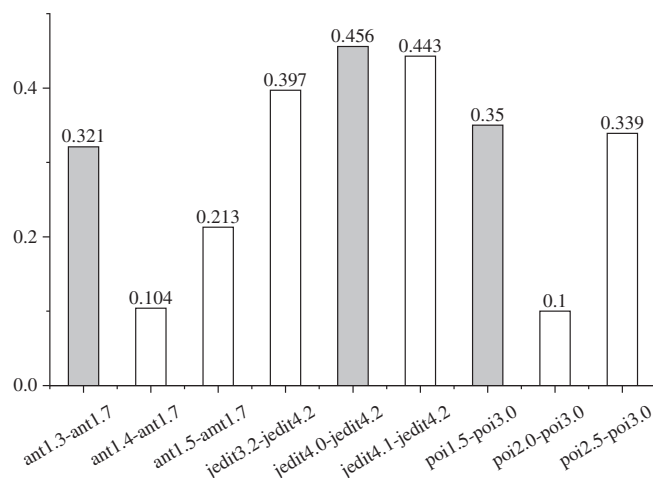


FIGURE 5 Experimental results of three target versions under MCC in cross-version defect prediction (CVDP)

Previous studies uncertainty quantification in the existing deep learning methods to solve such problems in related fields, for example, medical research (Abdar, Fahami, et al., 2021; Amersfoort et al., 2021). Uncertainty quantification is that, when the prediction model outputs a prediction result, it is necessary to explain how reliable the prediction is, (i.e., quantify the uncertainty of the prediction results) to provide a reference for participants (Convery et al., 2021). For example, due to the extreme sensitivity of the COVID-19, Abdar, Salari, et al. (2021) proposed that model uncertainty should be considered and customized a novel powerful fusion model named UncertaintyFuseNet using ensemble Monte Carlo (EMC) dropout to quantitatively evaluate the uncertainty of the output results. Although this technique has not been popularized in defect prediction, it would benefit both developers and testers if the technique can be adapted to work in software defect prediction. Also, transfer learning is a new learning paradigm in machine learning, which aims to solve the challenging learning problem with few or no labelled samples in the target domain (Pan et al., 2019; Wang et al., 2020; Wei et al., 2017). Prior studies have demonstrated the ability of instance-based transfer learning to solve such problems in other areas (Chen et al., 2019; Chowdhury et al., 2019; Edmonds et al., 2020). If these methods are feasible, how to transfer source data for the target version, how to reduce the impact of edge instances in the source domain, and how to set objective functions all will become new problems. In the future, we will continue to focus on this issue, and extend these alternative technologies to solve the pivotal problems in defect prediction.

7 | THREATS TO VALIDITY

Three main threats to validity in this section, that is, external validity, internal validity and construct validity, respectively, are illustrated as follows.

7.1 | External validity

Some external factors may impact the conclusions of our studies, including dataset, baseline, and model parameters. The approach presented in this paper has only been validated by the Jureczko dataset (Ferenc et al., 2020; Jureczko & Madeyski, 2010). The open-source project for this dataset uses object-oriented programming language (JAVA), and there are no other programming languages (C, C++, Python, and so on). The results of private projects and other projects with other programming languages are not clear. The baseline we set is composed of four defect models from four different types of algorithms. Replacing an independent model as a baseline will change our current statistical test results. However, this will also lose the objectivity and accuracy of the baseline.

7.2 | Internal validity

There are some internal threats in this paper. Firstly, Jureczko dataset (Jureczko & Madeyski, 2010) is used in the experiment, where the defect prediction features provided by Ferenc et al. (2020) are used. These features consist of 60 different kinds (size, complexity, coupling, cohesion,

inheritance, and documentation) of source code metrics for packages and class-level elements. If other features (process metrics and semantic information) are applied, better or worse prediction results may be obtained, which have not been validated. Secondly, in this study, we used parameters of the control models that are common in other studies, such as random forests and Adaboost, as described in Chapter 2. For Vote, Adaboost, and Bagging, the commonly used base learners are selected, and the performance of other base learners is not verified. Final, as we all know, there are many toolkits for machine learning, covering MATLAB, Scikit-learn, and Weka. We implement these algorithm models using the package provided by Weka, including baseline and control approaches, where the classifier we examined is not improved. When switching the technical route in this study, it is possible to obtain results that are slightly different from the present results.

7.3 | Construct validity

In the experiment, there are some structural threats, which come from two sources: performance evaluation and statistical results. Firstly, the class imbalance can reduce the accuracy and reliability of the defect model in CVDP and IVDP, as shown in the result analysis. And, class imbalances can distort some performance indicators about machine learning, for example, accuracy, precision, and F-measure (Yao & Shepperd, 2021). Based on practical experience, we employ two basic and three comprehensive indicators to evaluate the performance of defect models, improving the reliability of the evaluation results in CVDP and IVDP. Secondly, in the IVDP, we adopted the average results of 10-fold cross-validation 100 times to evaluate the performance of the model. Due to the different sample divisions, the evaluation results are also different, even some algorithms have only partial effective values 100 times, which is inevitable. Because its mean is not objective enough, compared with other algorithms, it will affect the experimental analysis and difference test. If this situation can be avoided, more objective and accurate conclusions may be obtained.

8 | SUMMARY AND PROSPECT

In software defect prediction, higher FPR will be produced due to class imbalance, while many solutions to class imbalances ignore this problem. For reducing the FPR, our WBN is a BayesNet with adjustable weights in this paper, which focuses more on false alarm instances. The principle of WBN is that when training the deficient model, the WBN changes the sampling probability of the misclassified instances through the classification error rate. To clarify the impact of the commonly used approaches to counteract class imbalance on FPR, the performance of AdaBoost, Bagging, Vote, ISDA, and random forest are evaluated in this paper, which are the contrasting methods we chose in the study. We set up two defect prediction scenarios (i.e., IVDP and CVDP) to verify the effectiveness of our WBN. The experimental results on eight open-source projects showed that the average value of the FPR of WBN decreased by 23.35% and 26.98%, respectively, compared with baseline in IVDP and CVDP. In IVDP, our WBN reduces FPR without significant difference from the contrasting defect models. And, in CVDP, our WBN not only reduces FPR, but also has significant superiority over the control defect model. Based on the experimental results of CVDP, we found that the differences in the class imbalance between the training set and test set can affect the reliability of the defect prediction. We recommend that practitioners select an appropriate training set from historical versions to predict the potential defects of the upcoming version, which has not been found in other studies.

In the future work, we will continue to focus on the pivotal problems in defect prediction, such as evaluating the uncertainty of prediction results of defect model and adapting training set for target version in CVDP.

ACKNOWLEDGEMENTS

This work was supported by the National Natural Science Foundation of China (No. U1736110) and Guangxi Key Laboratory of Cryptography and Information Security (No. GCIS202103).

CONFLICT OF INTEREST

The authors declare no potential conflict of interests.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study and source code for WBN and control methods are available in github repository. Please see: <https://github.com/YanYoungZhao/WBN-for-Defect-Prediction>.

ORCID

Yanyang Zhao  <https://orcid.org/0000-0002-1663-8901>

REFERENCES

- Abdar, M., Fahami, M. A., Chakrabarti, S., Khosravi, A., Pławiak, P., Acharya, U. R., Tadeusiewicz, R., & Nahavandi, S. (2021). Barf: A new direct and cross-based binary residual feature fusion with uncertainty-aware module for medical image classification. *Information Sciences*, 577, 353–378. <https://doi.org/10.1016/j.ins.2021.07.024>
- Abdar, M., Pourpanah, F., Hussain, S., Rezaadegan, D., Liu, L., Ghavamzadeh, M., Fieguth, P., Cao, X., Khosravi, A., Acharya, U. R., Makarencov, V., & Nahavandi, S. (2021). A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 76, 243–297. <https://doi.org/10.1016/j.inffus.2021.05.008>
- Abdar, M., Salari, S., Qahremani, S., Lam, H.-K., Karray, F., Hussain, S., Khosravi, A., Acharya, U. R., Makarencov, V., & Nahavandi, S. (2021). Uncertaintyfusenet: Robust uncertainty-aware hierarchical feature fusion with ensemble Monte Carlo dropout for covid-19 detection.
- Abdar, M., Samami, M., Dehghani Mahmoodabad, S., Doan, T., Mazouze, B., Hashemifesharaki, R., Liu, L., Khosravi, A., Acharya, U. R., Makarencov, V., & Nahavandi, S. (2021). Uncertainty quantification in skin cancer classification using three-way decision-based bayesian deep learning. *Computers in Biology and Medicine*, 135, 104418. <https://doi.org/10.1016/j.combiomed.2021.104418>
- Amersfoort, J. V., Smith, L., Jesson, A., Key, O., & Gal, Y. (2021). Improving deterministic uncertainty estimation in deep learning for classification and regression. CoRR, abs/2102.11409. <https://arxiv.org/abs/2102.11409>
- Arar, O. F., & Ayan, K. (2015). Software defect prediction using cost-sensitive neural network. *Applied Soft Computing*, 33, 263–277. <https://doi.org/10.1016/j.asoc.2015.04.045>
- Arun, C., & Lakshmi, C. (2020). Class imbalance in software fault prediction data set. In S. S. Dash, C. Lakshmi, S. Das, & B. K. Panigrahi (Eds.), *Artificial intelligence and evolutionary computations in engineering systems* (pp. 745–757). Springer Singapore.
- Bener, A., Misirli, A. T., Caglayan, B., Kocaguneli, E., & Calikli, G. (2015). Lessons learned from software analytics in practice. In C. Bird, T. Menzies, & T. Zimmermann (Eds.), *The art and science of analyzing software data* (pp. 453–489). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-411519-4.00016-1>
- Bennin, K. E., Keung, J., Phannachitta, P., Monden, A., & Mensah, S. (2018). Mahakil: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction. *IEEE Transactions on Software Engineering*, 44(6), 534–550. <https://doi.org/10.1109/TSE.2017.2731766>
- Bennin, K. E., Keung, J. W., & Monden, A. (2019). On the relative value of data resampling approaches for software defect prediction. *Empirical Software Engineering*, 24(2), 602–636. <https://doi.org/10.1007/s10664-018-9633-6>
- Bennin, K. E., Toda, K., Kamei, Y., Keung, J., Monden, A., & Ubayashi, N. (2016, August). *Empirical evaluation of cross-release effort-aware defect prediction models*. IEEE International Conference on Software Quality, Reliability and Security (QRS) (p. 214–221). doi: <https://doi.org/10.1109/QRS.2016.33>
- Chawla, N. V., Lazarevic, A., Hall, L. O., & Bowyer, K. W. (2003). Smoteboost: Improving prediction of the minority class in boosting. In N. Lavrač, D. Gamberger, L. Todorovski, & H. Blockeel (Eds.), *Knowledge discovery in databases: Pkdd 2003* (pp. 107–119). Springer Berlin Heidelberg.
- Chen, Q., Xue, B., & Zhang, M. (2019, June). Instance based transfer learning for genetic programming for symbolic regression (pp. 3006–3013). IEEE Congress on Evolutionary Computation (CEC). doi: <https://doi.org/10.1109/CEC.2019.8790217>
- Chowdhury, S. B. R., Annervaz, K. M., & Dukkupati, A. (2019). Instance-based inductive deep transfer learning by cross-dataset querying with locality sensitive hashing. In: C. Cherry, G. Durrett, G. F. Foster, R. Haffari, S. Khadivi, N. Peng, X. Ren & S. Swayamdipta, eds. *Proceedings of the Second Workshop on Deep Learning Approaches for Low-Resource NLP, Hong Kong, China*. 183–191. Association for Computational Linguistics. <https://aclanthology.org/D19-6120>
- Convery, O., Smith, L., Gal, Y., & Hanuka, A. (2021). Uncertainty quantification for virtual diagnostic of particle accelerators. *Physical Review Accelerators and Beams*, 24, 074602. <https://doi.org/10.1103/PhysRevAccel-Beams.24.074602>
- Diez-Pastor, J. F., Rodriguez, J. J., Garcia-Osorio, C., & Kuncheva, L. I. (2015). Random balance: Ensembles of variable priors classifiers for imbalanced data. *Knowledge-Based Systems*, 85, 96–111. <https://doi.org/10.1016/j.knosys.2015.04.022>
- Edmonds, M., Ma, X., Qi, S., Zhu, Y., Lu, H., & Zhu, S. (2020, February 7–12). *Theory-based causal transfer: Integrating instance-level induction and abstract-level structure learning*. The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, the Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, the Tenth AAAI Symposium on Educational Advances in Artificial Intelligence (pp. 1283–1291), AAAI 2020, AAAI Press. Retrieved from <https://aaai.org/ojs/index.php/AAAI/article/view/5483>
- Feng, S., Keung, J., Yu, X., Xiao, Y., Bennin, K. E., Kabir, M. A., & Zhang, M. (2021). Coste: Complexity-based oversampling technique to alleviate the class imbalance problem in software defect prediction. *Information and Software Technology*, 129, 106432. <https://doi.org/10.1016/j.infsof.2020.106432>
- Ferenc, R., Toth, Z., Ladanyi, G., Siket, I., & Gyimathy, T. (2020). A public unified bug dataset for java and its assessment regarding metrics and bug prediction. *Software Quality Journal*, 28, 1447–1506. <https://doi.org/10.1007/s11219-020-09515-0>
- Flach, P. A. (2003). *The geometry of roc space: Understanding machine learning metrics through roc isometrics* (pp. 194–201). Proceedings of the Twentieth International Conference on International Conference on Machine Learning. AAAI Press.
- Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139. <https://doi.org/10.1006/jcss.1997.1504>
- Galar, M., Fernandez, A., Barrenechea, E., Bustince, H., & Herrera, F. (2012). A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(4), 463–484. <https://doi.org/10.1109/TSMCC.2011.2161285>
- Ganganwar, V. (2012). An overview of classification algorithms for imbalanced datasets. *International Journal of Emerging Technology and Advanced Engineering*, 2, 42–47.
- Gao, H., Huang, W., Yang, X., Duan, Y., & Yin, Y. (2018). Toward service selection for workflow reconfiguration: An interface-based computing solution. *Future Generation Computer Systems*, 87, 298–311. <https://doi.org/10.1016/j.future.2018.04.064>
- Gao, H., Zhang, Y., Miao, H., Barroso, R. J. D., & Yang, X. (2021). Sdtioa: Modeling the timed privacy requirements of iot service composition: A user interaction perspective for automatic transformation from bpel to timed automata. doi: <https://doi.org/10.1007/s11036-021-01846-x>, 26, 2272, 2297
- Gao, K., Khoshgoftaar, T., & Napolitano, A. (2014). The use of ensemble-based data preprocessing techniques for software defect prediction. *International Journal of Software Engineering and Knowledge Engineering*, 24, 1229–1253. <https://doi.org/10.1142/S0218194014400105>
- Gu, Q., Zhu, L., & Cai, Z. (2009). Evaluation measures of the classification performance of imbalanced data sets. In Z. Cai, Z. Li, Z. Kang, & Y. Liu (Eds.), *Computational intelligence and intelligent systems* (pp. 461–471). Springer Berlin Heidelberg.
- Guo, H., & Viktor, H. L. (2004). Learning from imbalanced data sets with boosting and data generation: The databoost-im approach. *SIGKDD Explorations Newsletter*, 6(1), 30–39. Retrieved from <https://doi.org/10.1145/1007730.1007736>
- Hall, T., Beecham, S., Bowes, D., Gray, D., & Counsell, S. (2012). A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering*, 38(6), 1276–1304. <https://doi.org/10.1109/TSE.2011.103>

- Herbold, S. (2019). On the costs and profit of software defect prediction. *IEEE Transactions on Software Engineering*, 1-1, 2617–2631. <https://doi.org/10.1109/TSE.2019.2957794>
- Huang, Y., Xu, H., Gao, H., Ma, X., & Hussain, W. (2021). Ssur: An approach to optimizing virtual machine allocation strategy based on user requirements for cloud data center. *IEEE Transactions on Green Communications and Networking*, 5(2), 670–681. <https://doi.org/10.1109/TGCN.2021.3067374>
- Jing, X.-Y., Wu, F., Dong, X., & Xu, B. (2017). An improved sda based defect prediction framework for both within-project and cross-project class-imbalance problems. *IEEE Transactions on Software Engineering*, 43(4), 321–339. <https://doi.org/10.1109/TSE.2016.2597849>
- Jing, X. Y., Ying, S., Zhang, Z. W., Wu, S. S., & Liu, J. (2014, May). Dictionary learning based software defect prediction (pp. 414–423). ICSE 2014: Proceedings of the 36th International Conference on Software Engineering.
- Jureczko, M., & Madeyski, L. (2010). Towards identifying software project clusters with regard to defect prediction. Proceedings of the Sixth International Conference on Predictive Models in Software Engineering. Association for Computing Machinery. <https://doi.org/10.1145/1868328.1868342>
- Kamei, Y., Monden, A., Matsumoto, S., Kakimoto, T., & Matsumoto, K. I. (2007). The effects of over and under sampling on fault-prone module detection. First International Symposium on Empirical Software Engineering and Measurement. ESEM 2007. doi: <https://doi.org/10.1109/ESEM.2007.28>
- Khan, S. A., & Ali Rana, Z. (2019, February). Evaluating performance of software defect prediction models using area under precision-recall curve (auc-pr). Second International Conference on Advancements in Computational Sciences (ICACS) (p. 1–6). doi: <https://doi.org/10.23919/ICACS.2019.8689135>
- Khoshgoftaar, T. M., Gao, K., & Napolitano, A. (2014, August). Improving software quality estimation by combining feature selection strategies with sampled ensemble learning. Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014) (p. 428–433). doi: <https://doi.org/10.1109/IRI.2014.7051921>
- Khoshgoftaar, T. M., Geleyn, E., Nguyen, L., & Bullard, L. (2002, October). Cost-sensitive boosting in software quality modeling (pp. 51–60). Proceedings of Seventh IEEE International Symposium on High Assurance Systems Engineering. doi: <https://doi.org/10.1109/HASE.2002.1173102>
- Li, W., Meng, W., Su, C., & Kwok, L. F. (2018). Towards false alarm reduction using fuzzy if-then rules for medical cyber physical systems. *IEEE Access*, 6, 6530–6539. <https://doi.org/10.1109/ACCESS.2018.2794685>
- Li, Z., Jing, X., & Zhu, X. (2018). Progress on approaches to software defect prediction. *IET Software*, 12(3), 161–175. <https://doi.org/10.1049/iet-sen.2017.0148>
- Liu, M., Miao, L., & Zhang, D. (2014). Two-stage cost-sensitive learning for software defect prediction. *IEEE Transactions on Reliability*, 63(2), 676–686. <https://doi.org/10.1109/TR.2014.2316951>
- Lopez, V., Fernandez, A., Garcia, S., Palade, V., & Herrera, F. (2013). An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics. *Information Sciences*, 250, 113–141. <https://doi.org/10.1016/j.ins.2013.07.007>
- Luque, A., Carrasco, A., Martin, A., & de las Heras, A. (2019). The impact of class imbalance in classification performance metrics based on the binary confusion matrix. *Pattern Recognition*, 91, 216–231. <https://doi.org/10.1016/j.patcog.2019.02.023>
- Ma, X., Xu, H., Gao, H., & Bian, M. (2021). Real-time multiple-workflow scheduling in cloud environments. *IEEE Transactions on Network and Service Management*, 18(4), 4002–4018. <https://doi.org/10.1109/TNSM.2021.3125395>
- Ma, Y., Luo, G., & Chen, H. (2012). Kernel based asymmetric learning for software defect prediction. *IEICE Transactions on Information and Systems*, E95-D, 267–270. <https://doi.org/10.1587/transinf.E95.D.267>
- Mahmood, Z., Bowes, D., Lane, P. C. R., & Hall, T. (2015). What is the impact of imbalance on software defect prediction performance? Proceedings of the 11th International Conference on Predictive Models and Data analytics in Software Engineering; Association for Computing Machinery; New York, NY. Retrieved from <https://doi.org/10.1145/2810146.2810150>
- Menzies, T., Greenwald, J., & Frank, A. (2007). Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering*, 33(1), 2–13. <https://doi.org/10.1109/TSE.2007.256941>
- Menzies, T., Kocaguneli, E., Minku, L., Peters, F., & Turhan, B. (2015). How to keep your data private. In T. Menzies, E. Kocaguneli, L. Minku, F. Peters, & B. Turhan (Eds.), *Sharing data and models in software engineering* (pp. 165–196). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-417295-1.00016-3>
- Ozturk, M. M. (2017). Which type of metrics are useful to deal with class imbalance in software defect prediction? *Information and Software Technology*, 92, 17–29. <https://doi.org/10.1016/j.infsof.2017.07.004>
- Pan, W., Yang, Q., Cai, W., Chen, Y., Zhang, Q., Peng, X., & Ming, Z. (2019). Transfer to rank for heterogeneous one-class collaborative filtering. *ACM Transactions on Information Systems*, 37(1), 10:1–10:20. <https://doi.org/10.1145/3243652>
- Pandey, S. K., & Tripathi, A. K. (2021). An empirical study toward dealing with noise and class imbalance issues in software defect prediction. *Soft Computing*, 25(21), 13465–13492. Retrieved from <https://doi.org/10.1007/s00500-021-06096-3>
- Scott, A. J., & Knott, M. (1974). A cluster analysis method for grouping means in the analysis of variance. *Biometrics*, 30(3), 507–512. Retrieved from <https://doi.org/10.2307/2529204>
- Seiffert, C., Khoshgoftaar, T. M., & Van Hulse, J. (2009). Improving software-quality predictions with data sampling and boosting. *IEEE Transactions on Systems Man and Cybernetics - Part A*, 39(6), 1283–1294. <https://doi.org/10.1109/TSMCA.2009.2027131>
- Seiffert, C., Khoshgoftaar, T. M., Van Hulse, J., & Napolitano, A. (2010). Rusboost: A hybrid approach to alleviating class imbalance. *IEEE Transactions on Systems Man and Cybernetics - Part A: Systems and Humans*, 40(1), 185–197. <https://doi.org/10.1109/TSMCA.2009.2029559>
- Shull, F., Basili, V., Boehm, B., Brown, A. W., Costa, P., Lindvall, M., Port, D., Rus, I., Tvedt, R. T., & Zelkowitz, M. (2002a, June). What we have learned about fighting defects. Proceedings Eighth IEEE Symposium on Software Metrics (p. 249–258). doi: <https://doi.org/10.1109/METRIC.2002.1011343>
- Shull, F., Basili, V., Boehm, B., Brown, A. W., Costa, P., Lindvall, M., Port, D., Rus, I., Tvedt, R. T., & Zelkowitz, M. (2002b, June). What we have learned about fighting defects. Proceedings Eighth IEEE Symposium on Software Metrics (p. 249–258). doi: <https://doi.org/10.1109/METRIC.2002.1011343>
- Siers, M. J., & Islam, M. Z. (2015). Software defect prediction using a cost sensitive decision forest and voting, and a potential solution to the class imbalance problem. *Information Systems*, 51, 62–71. <https://doi.org/10.1016/j.is.2015.02.006>
- Song, Q., Guo, Y., & Shepperd, M. (2019, December). A comprehensive investigation of the role of imbalanced learning for software defect prediction. *IEEE Transactions on Software Engineering*, 45(12), 1253–1269. <https://doi.org/10.1109/TSE.2018.2836442>
- Sri Kavya, K. (2020). An ensemble deepboost classifier for software defect prediction. *International Journal of Advanced Trends in Computer Science and Engineering*, 9, 2021–2028. <https://doi.org/10.30534/ijatcse/2020/173922020>
- Sun, Z., Song, Q., & Zhu, X. (2012). Using coding-based ensemble learning to improve software defect prediction. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(6), 1806–1817. <https://doi.org/10.1109/TSMCC.2012.2226152>
- Tantithamthavorn, C., Hassan, A. E., & Matsumoto, K. (2020). The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering*, 46(11), 1200–1219. <https://doi.org/10.1109/TSE.2018.2876537>

- Tantithamthavorn, C., McIntosh, S., Hassan, A. E., & Matsumoto, K. (2016). *Automated parameter optimization of classification techniques for defect prediction models*. Proceedings of the 38th International Conference on Software Engineering (pp. 321–332). Association for Computing Machinery; New York, NY. Retrieved from <https://doi.org/10.1145/2884781.2884857>
- Tantithamthavorn, C., McIntosh, S., Hassan, A. E., & Matsumoto, K. (2017). An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering*, 43(1), 1–18. <https://doi.org/10.1109/TSE.2016.2584050>
- Thabtah, F., Hammoud, S., Kamalov, F., & Gonsalves, A. (2020). Data imbalance in classification: Experimental evaluation. *Information Sciences*, 513, 429–441. <https://doi.org/10.1016/j.ins.2019.11.004>
- Wahono, R. S. (2015). A systematic literature review of software defect prediction: Research trends, datasets, methods and frameworks. *Journal of Software Engineering*, 1(1), 1–16.
- Wang, J., Chen, Y., Feng, W., Yu, H., Huang, M., & Yang, Q. (2020). Transfer learning with dynamic distribution adaptation. *ACM Transactions on Intelligent Systems and Technology*, 11(1), 6:1–6:25. <https://doi.org/10.1145/3360309>
- Wang, J., Shen, B., & Chen, Y. (2012, August). *Compressed c4.5 models for software defect prediction*. 12th International Conference on Quality Software (p. 13–16). doi: <https://doi.org/10.1109/QSIC.2012.19>
- Wang, S., & Yao, X. (2013). Using class imbalance learning for software defect prediction. *IEEE Transactions on Reliability*, 62(2), 434–443. <https://doi.org/10.1109/TR.2013.2259203>
- Wang, T., & Li, W. (2010, December). *Naive Bayes software defect prediction model*. International Conference on Computational Intelligence and Software Engineering (p. 1–4). doi: <https://doi.org/10.1109/CISE.2010.5677057>
- Wang, T., Zhang, Z., Jing, X., & Zhang, L. (2016). Multiple kernel ensemble learning for software defect prediction. *Automated Software Engineering*, 23(4), 569–590. <https://doi.org/10.1007/s10515-015-0179-1>
- Wei, Y., Zhang, Y., & Yang, Q. (2017). Learning to transfer. CoRR, abs/1708.05629. Retrieved from <http://arxiv.org/abs/1708.05629>
- Witten, I. H., Frank, E., & Hall, M. A. (2011). *Data mining: Practical machine learning tools and techniques* (3rd ed., pp. 587–605). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-374856-0.00023-7>
- Xu, Z., Li, S., Luo, X., Liu, J., Zhang, T., Tang, Y., Xu, J., Yuan, P., & Keung, J. (2019). Tstss: A two-stage training subset selection framework for cross version defect prediction. *Journal of Systems and Software*, 154, 59–78. <https://doi.org/10.1016/j.jss.2019.03.027>
- Yao, J., & Shepperd, M. (2020). Assessing software defection prediction performance: Why using the Matthews correlation coefficient matters. Proceedings of the Evaluation and Assessment in Software Engineering (pp. 120–129). Association for Computing Machinery; New York, NY. Retrieved from <https://doi.org/10.1145/3383219.3383232>
- Yao, J., & Shepperd, M. (2021). The impact of using biased performance metrics on software defect prediction research. *Information and Software Technology*, 139, 106664. <https://doi.org/10.1016/j.infsof.2021.106664>
- Zhang, H., & Zhang, X. (2007). Comments on data mining static code attributes to learn defect predictors? *IEEE Transactions on Software Engineering*, 33(9), 635–637. <https://doi.org/10.1109/TSE.2007.70706>
- Zhang, X., Song, Q., Wang, G., Zhang, K., He, L., & Jia, X. (2015). A dissimilarity-based imbalance data classification algorithm. *Applied Intelligence*, 42, 544–565. <https://doi.org/10.1007/s10489-014-0610-5>
- Zhang, Y., Jin, D., Xing, Y., & Gong, Y. (2020). Automated defect identification via path analysis-based features with transfer learning. *Journal of Systems and Software*, 166, 110585. <https://doi.org/10.1016/j.jss.2020.110585>
- Zhang, Y., Xing, Y., Gong, Y., Jin, D., Li, H., & Liu, F. (2020). A variable-level automated defect identification model based on machine learning. *Soft Computing*, 24(2), 1045–1061. <https://doi.org/10.1007/s00500-019-03942-3>
- Zheng, J. (2010). Cost-sensitive boosting neural networks for software defect prediction. *Expert Systems with Applications*, 37(6), 4537–4543. <https://doi.org/10.1016/j.eswa.2009.12.056>

AUTHOR BIOGRAPHIES

Yanyang Zhao is currently a PhD candidate in the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing. She received her ME degree in Control Science and Engineering from Henan University of Technology in 2018. Her research interests include software testing and machine learning.

Yawen Wang received her PhD degree in Beijing University of Posts and Telecommunications, Beijing, in 2010. She is currently an associate professor in the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing. Her research interests include software testing and code analysis.

Dalin Zhang received his PhD degree in Beijing University of Posts and Telecommunications, Beijing, in 2014. He is currently an associate professor and supervisor of doctoral students in Beijing Jiaotong University. His research interests include software engineering services, software testing and artificial intelligence.

Yunzhan Gong received his PhD degree in computer science from Institute of Computing Technology, Chinese Academy of Sciences, Beijing, in 1991. He is currently a professor and supervisor of doctoral students in the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing. His research interests include fault tolerant computing and software testing.

How to cite this article: Zhao, Y., Wang, Y., Zhang, D., & Gong, Y. (2022). Eliminating the high false-positive rate in defect prediction through BayesNet with adjustable weight. *Expert Systems*, 39(6), e12977. <https://doi.org/10.1111/exsy.12977>